SMART LIBRARY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

RINU SWETHA S

(241401081)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

Computer Science & Business System



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Smart Library Management System” is the bonafide work of **RINU SWETHA S (241401081)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.,
HEAD OF THE DEPARTMENT
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

M. L. Alphin Ezhil Manuel,
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, **RINU SWETHA S (241401081)**, hereby declare that the project report titled “Smart Library Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, dashboard, Deluge scripts, workflows, Blueprint and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature, code snippet or external resource used in this work has been duly acknowledged in the report (see Section 5.9 for the AI assistance used).

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
RINU SWETHA S
(241401081)

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Mr. S. Meganathan, B.E., F.I.E.,** Chairperson **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K. R. Sowmia, Ph.D.,** Associate Professor & Head, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **M. L. Alphin Ezhil Manuel,** Assistant Professor (SG), for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

RINU SWETHA S

ABSTRACT

A college library depends on accurate records of its books, its members, the books issued and returned, the copies still available and the fines due. When these records are kept in registers or in unconnected spreadsheets, staff cannot easily tell how many copies of a title are on the shelf, which student holds a book, which transactions are overdue or how much fine is payable, and the fine has to be worked out by hand.

The Smart Library Management System is a low-code application built on Zoho Creator to centralise these activities. It has four linked forms, Book Details, Student Details, Book Transaction and Fine Payment, joined by lookup fields, including a multi-select Book lookup that lets one transaction cover several books, and an Additional Authors subform in Book Details. Reports and a Dashboard with summary cards, charts, year-wise statistics and Quick Links present the catalogue, student, transaction and fine information. Deluge scripts automate the library rules: an Open Library ISBN lookup through invokeurl fills the book name, publisher and publication year; available copies fall when a book is issued and are restored on return, with a Return Processed decision box that stops the same return from being counted twice; and issuing is restricted when no copy is available. Overdue fines are calculated at ₹50 per overdue day by the Calculate_Fine function and by a scheduled batch workflow, Bulk_Overdue_Fine_Update. A transaction e-mail and a daily overdue reminder keep users informed, a Blueprint named Book Issue Process guides each transaction through its lifecycle, and Admin, Librarian and Student roles with permission settings control access.

The application was tested while it was built. ISBN 9783161484100 returned “Logik der Forschung”, Mohr Siebeck, 2005; stock fell by one when a book was issued; available copies rose from 10 to 11 on return, and saving the same returned transaction again did not increase the stock a second time. The scheduled fine execution, the transaction e-mail, the daily overdue reminder, the Blueprint and the role permissions were run and verified. Duplicate availability workflows that caused errors during debugging were disabled and the intended no-copies workflow was retained. No timing measurements were recorded, so no quantitative benefit is claimed.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Library Management; Workflow Automation; Blueprint; Role-Based Access Control

PROJECT AT A GLANCE

Attribute	Details
Project Title	Smart Library Management System
Application Domain	Library catalogue, student records, circulation, stock, overdue fines and fine payments in an educational institution
Student Name & Register No.	RINU SWETHA S – 241401081
Zoho Creator Edition	Education account
Application Name (Link Name)	college-library-management-system
Published / Live URL	https://creatorapp.zoho.in/rinuswetha.s.2024.csbs_ra113/college-library-management-system
Demo Credentials (per role)	Admin, Librarian and Student logins are provided to the examiner at the demonstration (passwords are not printed in this report)
Number of Forms / Subforms	4 (Book Details, Student Details, Book Transaction, Fine Payment) / 1 (Additional Authors in Book Details)
Number of Reports (by type)	Book Details Report, Categories, All Student Details, Departments, All Book Transactions, All Fine Payments Total: 6 reports
Number of Pages / Dashboards	1 (Dashboard: summary cards, charts, year-wise statistics, Quick Links)
Number of Workflows	Form workflows: Fetch_Book_Details_API, Auto-Update Available Copies, Block Issuance If No Copies, return processing, transaction e-mail · Schedule: Bulk_Overdue_Fine_Update, daily overdue reminder · Blueprint: Book Issue Process
Deluge Functions / Custom Actions	Calculate_Fine(due_date, return_date) connected to Process Book Return, plus the form and schedule scripts listed above
Integrations	Open Library ISBN API (Deluge invokeurl, HTTP GET); e-mail notifications
Roles & Profiles Defined	Roles: Admin, Librarian, Student · Permission profiles: role-based, for controlled access to application features
Environments Used	Stage (version 1.0) and Production (version 1.0)
Date of Production Deployment	01-Nov-2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY .....
2.1 Study of the Existing System .....
2.2 Limitations of the Existing System .....
2.3 Low-Code versus Traditional Development .....
2.4 Literature Survey .....
2.5 Emerging Trends in Low-Code Development .....
2.6 Proposed System .....
REQUIREMENT ANALYSIS .....
3.1 Stakeholders and User Roles .....
3.2 Functional Requirements .....
3.3 Non-Functional Requirements .....
3.4 Hardware and Software Requirements .....
3.5 Use Case Model .....
SYSTEM DESIGN .....
4.1 System Architecture .....
4.2 Data Model .....
4.3 Form Design .....
4.4 Process and Workflow Design .....
4.5 User Interface and Navigation Design .....
4.6 Access Control Design .....
IMPLEMENTATION .....
5.1 Application Setup .....
5.2 Forms and Data Capture .....
5.3 Reports .....
5.4 Pages and Dashboards .....
5.5 Deluge Scripting .....
5.6 Workflows and Automation .....

5.7 Integrations .....
5.8 User Management and Role-Based Access .....
5.9 AI and Analytics Features .....
5.10 Pro-Code Extension .....
5.11 Mobile and Portal Access .....
TESTING AND DEBUGGING .....
6.1 Testing Strategy .....
6.2 Test Cases .....
6.3 Debugging Deluge Scripts .....
6.4 Defect Log .....
6.5 User Acceptance Testing .....
DEPLOYMENT, SECURITY AND GOVERNANCE .....
7.1 Environments and Deployment Strategy .....
7.2 Performance Optimisation .....
7.3 Security Features .....
7.4 Governance and Administration .....
RESULTS AND DISCUSSION .....
8.1 Application Output .....
8.2 Achievement of Objectives .....
8.3 Comparison with the Existing System .....
8.4 Limitations of the Low-Code Approach .....
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum .....
CONCLUSION AND FUTURE ENHANCEMENTS .....
9.1 Conclusion .....
9.2 Future Enhancements .....
9.3 Learning Outcomes and Career Relevance .....
REFERENCES .....
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST .....
APPENDIX B – COURSE OUTCOME MAPPING .....
APPENDIX C – COMPLETE DELUGE SOURCE CODE .....
APPENDIX D – USER MANUAL .....
D.1 Accessing the Application .....
D.2 Instructions for the Admin .....
D.3 Instructions for the Librarian .....
D.4 Instructions for the Student .....
APPENDIX E – EVALUATION RUBRIC .....
APPENDIX F – VIVA-VOCE RECORD SHEET .....

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Library Management Application .....
Table 2.2: Summary of Literature Survey .....
Table 3.1: Stakeholders and User Roles .....
Table 3.2: Functional Requirements .....
Table 3.3: Non-Functional Requirements .....
Table 3.4: Hardware and Software Requirements .....
Table 3.5: Use Case Description – Issue Book (UC-03) .....
Table 3.6: Use Case Description – Return Book (UC-04) .....
Table 4.1: Entities and Relationships .....
Table 4.2: Field Specification – Book Details Form .....
Table 4.3: Field Specification – Student Details Form .....
Table 4.4: Field Specification – Book Transaction Form .....
Table 4.5: Field Specification – Fine Payment Form .....
Table 4.6: Book Issue Process Blueprint – Stages and Transitions .....
Table 4.7: Role–Permission Matrix .....
Table 5.1: Reports Implemented .....
Table 5.2: Deluge Scripts Implemented .....
Table 5.3: Workflows Implemented .....
Table 5.4: Integrations Implemented .....
Table 6.1: Test Cases .....
Table 6.2: Defect Log .....
Table 6.3: User Acceptance Feedback .....
Table 7.1: Deployment Checklist .....
Table 7.2: Performance Optimisation Results .....
Table 7.3: Security Controls Implemented .....
Table 8.1: Mapping of Objectives to Outcomes .....
Table 8.2: Existing System versus Proposed Application .....
Table 8.3: Observed Application State from the Screenshots .....
Table A.1: Feature Coverage Checklist .....
Table B.1: Mapping of Course Outcomes to Report Chapters .....
Table E.1: Evaluation Rubric for the Individual Project .....
Table E.2: Performance Level Descriptors .....
Table F.1: Viva-Voce Questions and Responses .....
Table F.2: Marks Summary .....

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of Manual Library Record Keeping .....
Figure 3.1: Use Case Diagram of the Smart Library Management System .....
Figure 4.1: System Architecture of the Smart Library Management System .....
Figure 4.2: Entity–Relationship Diagram of the Application .....
Figure 4.3: Workflow / Process Flow for Issuing a Book .....
Figure 4.4: Workflow / Process Flow for Returning a Book and Restoring Stock .....
Figure 4.5: Lifecycle of a Transaction in the Book Issue Process Blueprint .....
Figure 4.6: Navigation Structure of the Application .....
Figure 5.1: Application Home and Navigation Menu .....
Figure 5.2: Book Details Form in the Form Builder .....
Figure 5.3: Additional Authors Subform Configuration .....
Figure 5.4: Student Details Form in the Form Builder .....
Figure 5.5: Book Transaction Form in the Form Builder .....
Figure 5.6: Fine Payment Form in the Form Builder .....
Figure 5.7: Book Details Form in Live Mode (ISBN Autofill) .....
Figure 5.8: Student Details Form in Live Mode .....
Figure 5.9: Book Transaction Form in Live Mode .....
Figure 5.10: Fine Payment Form in Live Mode .....
Figure 5.11: Book Details Report and Categories .....
Figure 5.12: All Student Details Report and Departments .....
Figure 5.13: All Book Transactions Report .....
Figure 5.14: All Fine Payments Report .....
Figure 5.15: Dashboard – Summary Cards and Charts .....
Figure 5.16: Dashboard – Year-wise Statistics and Quick Links .....
Figure 5.17: Workflow Fetch_Book_Details_API and Successful ISBN Result .....
Figure 5.18: Workflow Auto-Update Available Copies and Stock Before/After .....
Figure 5.19: Return Workflow and the return_processed Field .....
Figure 5.20: Workflow Block Issuance If No Copies .....
Figure 5.21: Calculate_Fine Function and Process Book Return .....
Figure 5.22: Scheduled Workflow Bulk_Overdue_Fine_Update .....
Figure 5.23: Transaction E-mail Workflow and Received E-mail .....
Figure 5.24: Daily Overdue Reminder Workflow .....
Figure 5.25: Book Issue Process Blueprint – Stages and Transitions .....
Figure 5.26: Roles in the Application .....
Figure 5.27: Permission Profiles for Admin, Librarian and Student .....
Figure 6.1: Duplicate Availability Workflows Found During Debugging .....
Figure 6.2: Repeated Save of a Returned Transaction Does Not Change Stock .....

Figure 7.1: Environment and Publish Settings of the Application .....
Figure 7.2: Administration of Users, Roles and Permissions .....
Figure 8.1: Scenario Step 1 – Book Issued to a Student .....
Figure 8.2: Scenario Step 2 – Available Copies Reduced .....
Figure 8.3: Scenario Step 3 – Book Returned and Stock Restored .....
Figure 8.4: Scenario Step 4 – Overdue Fine Updated and Dashboard Reflecting the Change .....

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSBS	Computer Science and Business Systems
DS	Deluge Script (script ID used in this report)
ER	Entity–Relationship
FR / NFR	Functional / Non-Functional Requirement
HTTP	Hypertext Transfer Protocol
ISBN	International Standard Book Number
JSON	JavaScript Object Notation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFID	Radio-Frequency Identification
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 Background

A library in an educational institution is one of the busiest record-keeping points on the campus. Every day it must answer a few simple questions correctly: which books it owns, how many copies of each title are on the shelf, which student has borrowed a book, when the book is due back, and whether any fine is payable. Behind these questions are four kinds of information: the catalogue of books, the records of the students who borrow them, the issue and return transactions, and the fines and fine payments that arise from late returns.

In many institutions these records are still kept in registers or in separate spreadsheets. The number of available copies must be adjusted by hand after every issue and return, overdue books are noticed only when somebody goes through the register, and the fine for a late return is worked out manually. A mistake in any one of these steps spoils the others, and nobody has a single place from which the whole position of the library can be seen.

The Smart Library Management System is a low-code application built on Zoho Creator to replace this practice. Books and students are registered once, and each issue or return is recorded as a transaction linked to the student and to the books concerned. The number of available copies is updated automatically, overdue fines are calculated at ₹50 per overdue day, fine payments are recorded against the transaction, and a dashboard shows the state of the library. The application was developed as an individual project in a Zoho Creator Education account.

1.2 Problem Statement

Library operations need accurate records of the book catalogue, the student borrowers, the issue and return transactions, the available copies, the overdue fines and the fine payments. When these activities are handled through disconnected or manual records, it becomes difficult to identify the correct book and borrower, to track issue and due dates, to keep the stock consistent, to find overdue transactions, to calculate fines and to monitor library activity as a whole. The main pain points are:

- Book details such as title, publisher and publication year are typed in by hand, which is slow and error-prone.
- Available copies are not updated consistently after an issue or a return, and a return can be counted more than once.
- A book can be issued even when no copy is left on the shelf.
- Overdue transactions are not identified promptly, and the fine for each overdue day is calculated manually.
- Fine payments are not linked to the transaction they belong to.

- There is no single view of the catalogue, the borrowers, the transactions and the fines, and every person who can see the records can change all of them.

1.3 Objectives

The project set out to achieve the following objectives:

- To design a data model of four linked forms (Book Details, Student Details, Book Transaction and Fine Payment) with lookup relationships, a multi-select Book lookup and an Additional Authors subform.
- To retrieve the book name, publisher and publication year automatically from the Open Library ISBN API through a Deluge invokeurl call.
- To keep the stock accurate: decrease available copies when a book is issued, restore them once on return with a duplicate-processing guard, and restrict issuing when no copies are available.
- To calculate overdue fines at ₹50 per overdue day through the Calculate_Fine function and the scheduled batch workflow Bulk_Overdue_Fine_Update.
- To maintain fine payment details linked to the relevant book transaction.
- To automate communication through a transaction e-mail and a daily overdue reminder.
- To represent the issue, return and fine lifecycle of a transaction through the Book Issue Process Blueprint.
- To provide reports and a dashboard with summary cards, charts, year-wise statistics and Quick Links.
- To control access through Admin, Librarian and Student roles and their permission settings.

1.4 Scope of the Project

In scope. The implemented application covers book catalogue management, student management, book transactions, fine payments, reporting and a dashboard. It contains four forms with an Additional Authors subform, reports for books, categories, students, departments, book transactions and fine payments, one dashboard, Deluge form and scheduled workflows, the Calculate_Fine function, an external REST API call to Open Library, a transaction e-mail, a daily overdue reminder, the Book Issue Process Blueprint and a three-role security model (Admin, Librarian, Student). The Open Library call fills the book name, publisher and publication year; the author is entered manually. The application is used through a web browser.

Out of scope. The following are not part of the delivered application and are labelled as proposed wherever they appear: barcode or RFID scanning of books and member cards, an online payment gateway for fines, expanded mobile and portal access, book recommendations and further analytics. A Kanban report was considered but is not part of the implemented report inventory. The Open Library integration is an external

REST API call made through invokeurl; no webhook or Creator REST API endpoint was built.

1.5 Justification for a Low-Code Approach

A library register is a data-entry and workflow application: a few related forms, some reports, a handful of calculations and a few notifications. These are the building blocks that a low-code application platform provides as configurable components. On Zoho Creator the four forms and their lookup relationships, the reports and the dashboard were assembled with the form builder, the report builder and the page builder, while Deluge scripting was needed only for the rules that forms alone cannot express: the ISBN lookup, the stock update on issue and return, the fine calculation and the scheduled batch. Hosting, authentication, user management, record storage and report rendering are supplied by the platform, so the effort could be concentrated on library logic instead of infrastructure. Published studies of low-code practice report that practitioners value the ease of learning and the speed of development of such platforms, while also reporting limits in customisation [1], [2].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The usual arrangement for running a college library without a system is manual. A student brings a book to the counter, and the librarian searches for the title and notes the student's name, the book and the dates in an issue register or a spreadsheet. The number of copies of the title is changed by hand in a stock register, the due date is written next to the entry, and nothing reminds anybody when that date passes. When the book comes back, the entry is found again, marked as returned and the stock is corrected by hand; if the book came back late, the number of overdue days is counted and the fine is worked out and recorded separately, often on paper. When the position of a title or of a student has to be found, the staff read through the register page by page. Figure 2.1 summarises this commonly followed practice. The description is a general account of manual library record keeping and is not a survey of one named library.

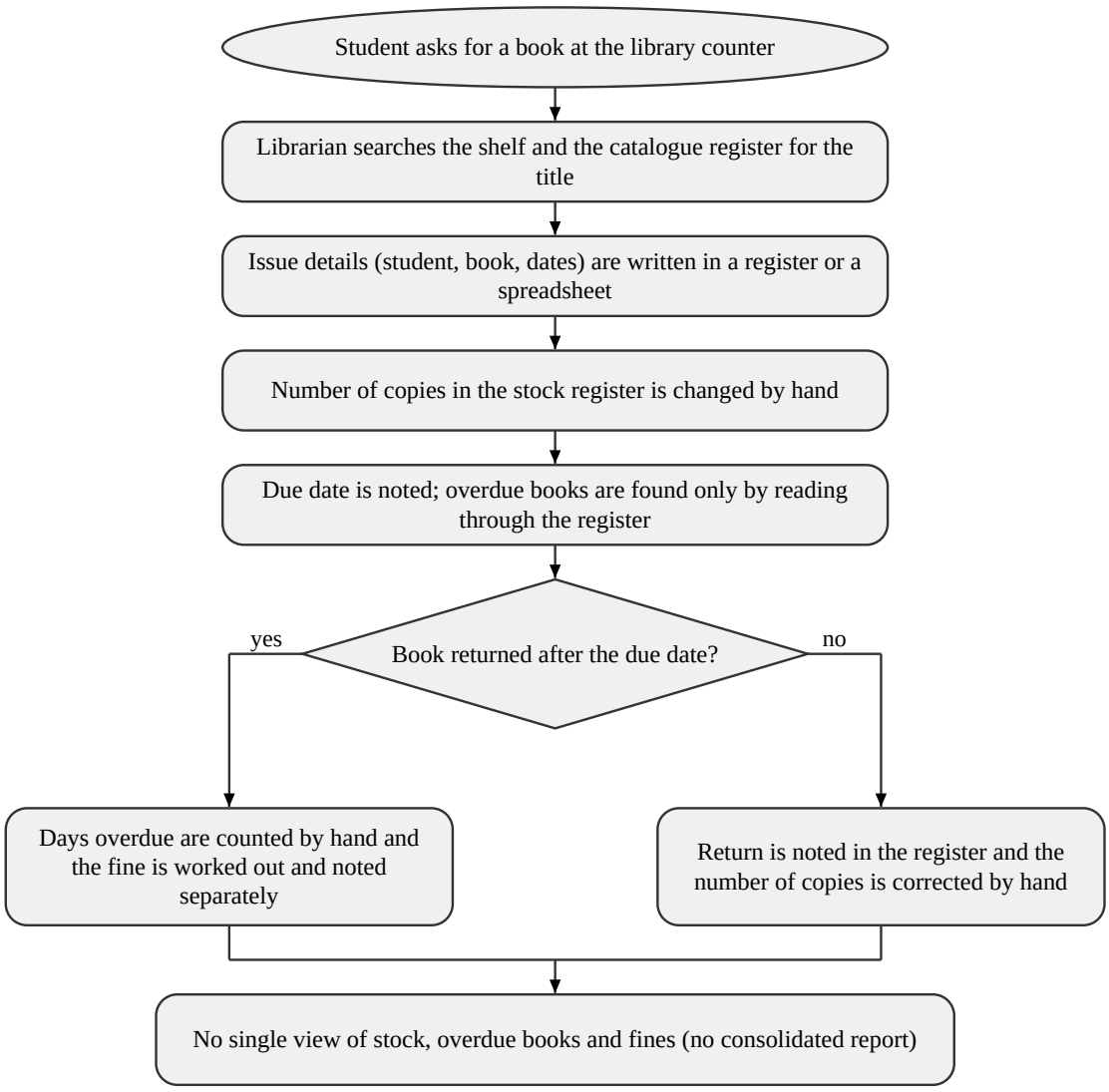


Figure 2.1: As-Is Process Flow of Manual Library Record Keeping

2.2 Limitations of the Existing System

- **Fragmented records.** Catalogue, borrower, issue and fine details are spread over registers and separate sheets, and are easily lost, duplicated or entered in different ways.
- **Manual book details.** The title, publisher and publication year are typed in for every new book, which is slow and leads to spelling and entry errors.
- **Stock mismatch.** Available copies are corrected by hand after each issue and return, so the count drifts away from the shelf. A return recorded twice inflates the stock, and a book may be issued when no copy is left.
- **Delayed overdue identification.** Overdue transactions are found only when somebody reads through the register, and no reminder is sent to the borrower.
- **Manual fine calculation.** The fine for each overdue day is counted by hand, so it can be inconsistent, and the payment is not tied to the transaction that caused it.
- **No lifecycle control.** There is no defined sequence of steps for approval, issue, return, fine, lost book or replacement; each case is handled informally.
- **Weak visibility.** Totals such as the number of titles, copies available, books on loan or fines due cannot be seen without counting by hand.
- **No access control.** Whoever holds the register or the spreadsheet can see and change everything in it.

2.3 Low-Code versus Traditional Development

Table 2.1 compares a traditional pro-code implementation of the library system with the low-code implementation on Zoho Creator. The comparison is qualitative. No development-time or cost measurements were recorded during the project, so none are claimed.

Table 2.1: Comparison of Low-Code and Traditional Development for the Library Management Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Database design, back-end services, user interface, a scheduler and a deployment pipeline must each be built and integrated.	Forms, reports and the dashboard are assembled in builders; Deluge scripts were written only for the ISBN lookup, stock update, fine calculation and scheduled processing.
Skills required	A front-end and a back-end language, SQL, a job scheduler, hosting and security administration.	Familiarity with the platform and Deluge scripting.
Hosting and infrastructure	Servers or cloud services, a database and an e-mail service must be provisioned, secured and maintained.	Hosted and maintained by the platform; the application runs inside a Zoho Creator account.
External API integration	HTTP client code, JSON parsing and error handling must be written and deployed.	A Deluge invokeurl call to the Open Library ISBN endpoint, with the JSON response parsed in the script.

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Workflow and scheduling	Cron jobs or a task queue, and a state machine for the transaction lifecycle, must be coded.	Form workflows, scheduled workflows and a Blueprint are configured in the Workflow builder.
Security and access control	Authentication, sessions and role logic must be implemented in code.	Admin, Librarian and Student roles and their permission settings are configured in the application settings.
Maintenance and change	Changes need code changes, testing and redeployment.	Fields, reports and workflows are edited in the builders; scripts in the Deluge editor.
Customisation limits	Practically unlimited.	Bounded by platform components and Deluge capabilities (see Section 8.4).
Cost	Developer time plus infrastructure and licences.	Platform subscription; this project was built in a Zoho Creator Education account.

2.4 Literature Survey

Table 2.2 summarises the sources used. Two are published empirical studies of low-code practice, and three are entries for the vendor documentation of the platform features and of the external service used in this project. Only the points that the cited sources are known to cover are recorded.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021 [1]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Empirical study of practitioners' discussions on low-code development, covering its advantages and its challenges.	Supports the choice of a low-code platform and the limitations discussion in Section 8.4.
2	M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, 2021 [2]	An Empirical Study of Developer Discussions on Low-Code Software Development Challenges	Empirical study of developer questions about low-code platforms, grouped into topics of challenges.	Background for the limitations and workarounds discussed in Sections 8.4 and 8.5.
3	Zoho Corporation [3]	Zoho Creator Help	Documentation of forms, reports, pages, workflows, Blueprints, connections, roles and permissions.	Reference for Chapters 4 and 5.
4	Zoho Corporation [4]	Deluge Script Reference	Documentation of the Deluge scripting language used in Creator workflows, including invokeurl.	Reference for the scripts in Section 5.5 and Appendix C.
5	Open Library [5]	Open Library Books API	Public documentation of how book data can be retrieved from Open Library by ISBN in JSON form.	Reference for the ISBN autofill described in Section 5.7.

2.5 Emerging Trends in Low-Code Development

Four general trends are relevant to this project. First, low-code platforms are becoming a normal way to build departmental business applications, and their builders increasingly include AI assistance; the logic of this project, however, is rule-based Deluge scripting written and checked by the student. Second, workflow automation is moving from manual follow-up to rule-driven processing, which this project applies through form workflows, a scheduled fine update, a daily overdue reminder and a Blueprint that guides each transaction through its lifecycle. Third, platforms emphasise integration with outside services through APIs, which this project exercises by calling the Open Library ISBN service from a Deluge script. Fourth, as more business users build applications, governance concerns such as role design and permission control become more important; this project applies them through Admin, Librarian and Student roles and their permission settings. These trends are stated qualitatively and are not backed by market figures.

2.6 Proposed System

The proposed system is the Smart Library Management System. Books and students are registered once in the Book Details and Student Details forms; when an ISBN is entered, the book name, publisher and publication year are fetched from Open Library, and the author is entered manually. Each issue or return is recorded in the Book Transaction form, which is linked to the student and, through a multi-select lookup, to one or more books. Available copies fall automatically when a book is issued and rise once when it is returned, issuing is restricted when no copy is available, and the fine is calculated at ₹50 per overdue day by the Calculate_Fine function and by a scheduled batch update. Fine payments are recorded in the Fine Payment form against the transaction. A transaction e-mail and a daily overdue reminder inform users, the Book Issue Process Blueprint guides the transaction through its stages, and Admin, Librarian and Student roles restrict access. Reports and a dashboard show the position of the library at a glance. The system therefore removes the scattered records and the manual book entry (Section 2.2, first and second points), keeps the stock consistent, identifies overdue transactions and calculates fines automatically, links each payment to its transaction, gives every transaction a defined lifecycle, summarises the data and restricts access.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

The Smart Library Management System is used by three groups of people, each represented by a role in the application. Table 3.1 lists the roles and their key responsibilities. The Open Library ISBN service and the e-mail service act only as supporting external parties and are treated as secondary actors in Section 3.5.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Application and library administrator; owner of the Zoho Creator application.	Configure forms, reports, workflows, the Blueprint, roles and permissions; oversee all library records; view the dashboard and all reports.
Librarian	Staff member who performs the day-to-day library operations.	Maintain the book catalogue and student records as permitted; issue and return books; handle overdue fines and fine payments; view reports and the dashboard.
Student	Library member and borrower; end user of the library.	View available books and own borrowing history as permitted; cannot administer library records or application settings.

3.2 Functional Requirements

Table 3.2 lists the functional requirements. FR-01 to FR-17 correspond to features built in the application; FR-18 and FR-19 are proposed requirements that have not been implemented. The IDs are used again in the test cases of Chapter 6.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall store book catalogue records in the Book Details form with ISBN, Book Name, Publisher, Publication Year, Author, Category, Book Code, Total Copies and Available Copies.	Administrator, Librarian	High
FR-02	The system shall fill Book Name, Publisher and Publication Year automatically from the Open Library ISBN service when an ISBN is entered (Fetch_Book_Details_API); the Author is entered manually.	Administrator, Librarian	High
FR-03	The system shall record additional authors of a book through the Additional Authors subform of Book Details.	Administrator, Librarian	Medium

FR ID	Requirement	Role(s)	Priority
FR-04	The system shall store student (member) records in the Student Details form, which is the source of the Student lookup used in transactions.	Administrator, Librarian	High
FR-05	The system shall record a book transaction with a multi-select Book lookup, a Student lookup, Status, Issue Date, Due Date, Return Date and Fine Amount.	Librarian, Administrator	High
FR-06	The system shall decrease Available Copies by one for each selected book when a transaction with status Issued is added (Auto-Update Available Copies).	System	High
FR-07	The system shall restrict the issue of a book when no copies are available (Block Issuance If No Copies).	System	High
FR-08	The system shall increase Available Copies by one for each selected book when a transaction is marked Returned, and shall do so only once by using the Return Processed decision box (return_processed).	System	High
FR-09	The system shall calculate the overdue fine at ₹50 per overdue day using the Calculate_Fine(due_date, return_date) function connected to Process Book Return.	System	High
FR-10	The system shall update the fine of overdue Issued transactions on a schedule (Bulk_Overdue_Fine_Update, batch size 1000) as overdue days × ₹50.	System	High
FR-11	The system shall store fine payment details (Transaction, Fine Amount, Payment Status, Payment Date, Payment Method) in the Fine Payment form, linked to the Book Transaction.	Librarian, Administrator	High
FR-12	The system shall send a transaction e-mail notification through the configured transaction email workflow.	System	Medium
FR-13	The system shall send a daily reminder for overdue transactions through the configured scheduled reminder workflow.	System	Medium
FR-14	The system shall represent the lifecycle of a transaction (initiation, approval, issue, use, fine, return, payment, closure, cancellation, rejection, lost book and replacement) through the Book Issue Process Blueprint.	Librarian, Administrator	Medium
FR-15	The system shall provide reports and menu entries for books (Book Details Report, Categories), students (All Student Details, Departments), transactions (All Book Transactions) and fine payments (All Fine Payments).	All roles (as permitted)	Medium
FR-16	The system shall provide a dashboard with summary cards, charts, year-wise statistics and Quick Links.	Administrator, Librarian	Medium

FR ID	Requirement	Role(s)	Priority
FR-17	The system shall restrict users through the Admin, Librarian and Student roles and their permission settings.	Administrator	High
FR-18	(Proposed) The system shall support barcode or RFID scanning for issuing and returning books.	Librarian	Proposed
FR-19	(Proposed) The system shall accept online fine payment through a payment gateway.	Student	Proposed

3.3 Non-Functional Requirements

Table 3.3 lists the non-functional requirements. No timing measurements were recorded for the application, so the performance requirement is stated qualitatively.

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Forms, reports and the dashboard shall open without perceptible delay for the small record volumes of the project. No measurement was recorded.
NFR-02	Security	Users shall reach only the forms, reports and pages allowed by their role (Admin, Librarian or Student) and its permission settings; passwords and private credentials shall not be stored in or printed from the application records.
NFR-03	Usability	Books and students shall be selected from lookups, book details shall be filled from the ISBN where available, and the application shall run in a standard web browser.
NFR-04	Accuracy	Available Copies shall stay consistent with issue and return activity: stock shall not go below zero on issue, and a returned transaction shall restore stock only once. The fine shall be the total overdue days × ₹50 and shall not be added cumulatively on each scheduled run.
NFR-05	Availability	The application relies on the availability of the Zoho Creator service and the internet connection of the user; no separate hosting is maintained. The ISBN autofill additionally depends on the Open Library service.
NFR-06	Maintainability	Each business rule shall be placed in a separately named workflow or function (for example Auto-Update Available Copies, Block Issuance If No Copies, Calculate_Fine and Bulk_Overdue_Fine_Update) so that it can be changed without redesigning the forms.
NFR-07	Interoperability	The ISBN lookup shall use an HTTP GET call (Deluge invokeurl) to the Open Library endpoint and read the JSON response; this is an external REST API call and not a webhook.
NFR-08	Auditability	Record changes are traceable only through the platform’s own record information; a dedicated audit-trail configuration is not claimed.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Platform	Zoho Creator (Education account)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	A current web browser such as Google Chrome
External service	Open Library ISBN API (https://openlibrary.org/isbn/{ISBN}.json), called with Deluge invokeurl
Notifications	E-mail messages sent by the transaction email workflow and the daily overdue reminder workflow
Mobile and portal	No separate mobile app or customer portal is claimed; the application is used through the web browser
Client hardware	Any computer able to run a current web browser, with an internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases. The Librarian maintains the catalogue and student records, issues and returns books, handles fines and fine payments, and views the reports and dashboard. The Administrator manages roles and permissions and also performs the Librarian use cases, as indicated by the dashed generalisation arrow, which is consistent with the administrator owning the application. The Student views the permitted book and borrowing information. The Open Library service appears as a secondary actor because it supplies the book details for the ISBN autofill, and the e-mail service appears as a secondary actor because it delivers the transaction e-mail and the daily overdue reminder.

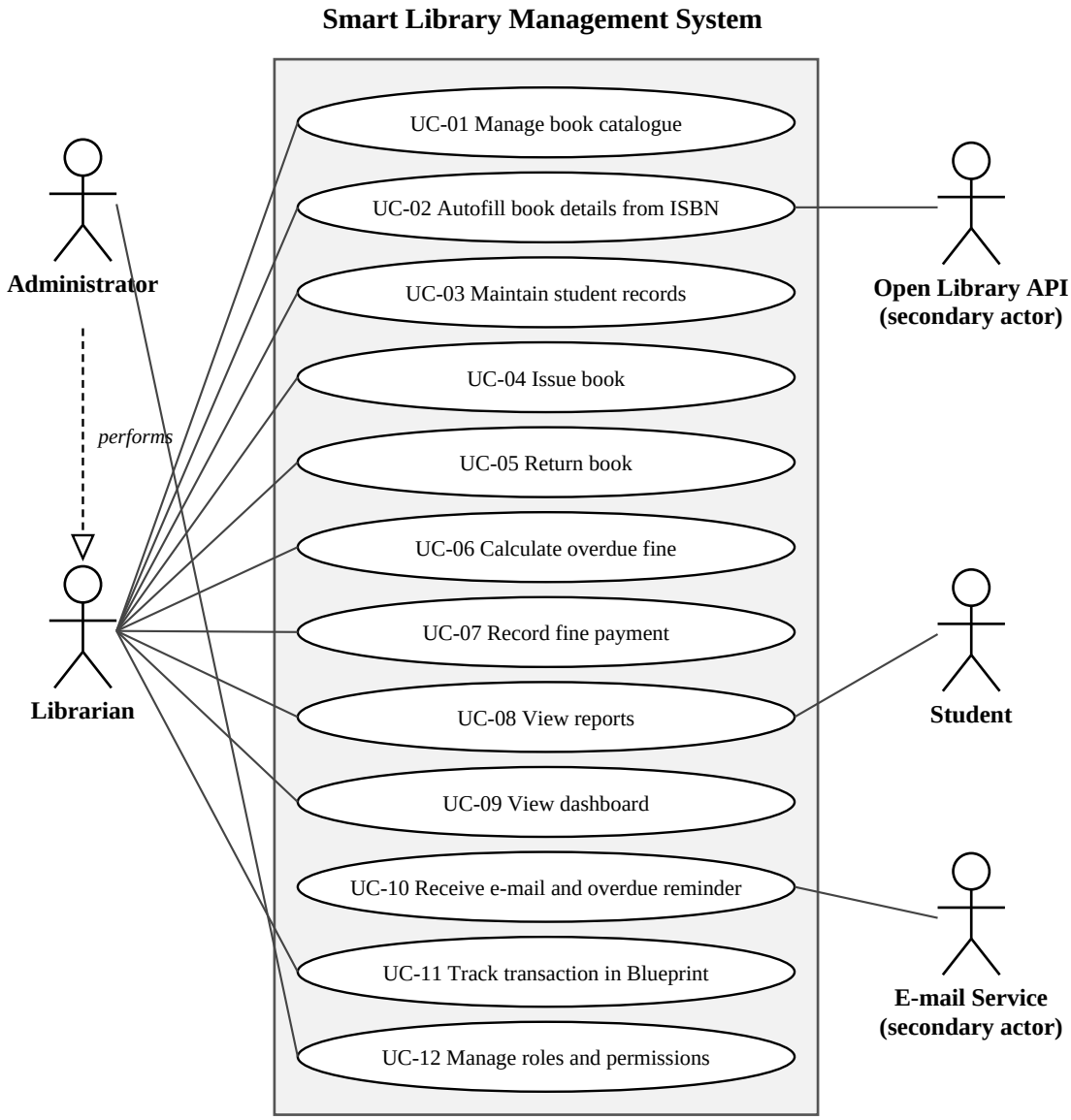


Figure 3.1: Use Case Diagram of the Smart Library Management System

Tables 3.5 and 3.6 describe two key use cases in detail.

Table 3.5: Use Case Description – Issue Book (UC-04)

Use Case ID	UC-04
Use Case Name	Issue Book
Actor(s)	Librarian (primary); Administrator
Pre-condition	The book is available in Book Details with Available Copies greater than zero, and the student is registered in Student Details; the actor has access to the Book Transaction form.

Main Flow	1. The actor opens Book Transactions and adds a new transaction. 2. The actor selects one or more books in the multi-select Book lookup and selects the Student. 3. The actor sets Status to Issued and enters the Issue Date and the Due Date. 4. The actor clicks Submit. 5. The Auto-Update Available Copies workflow loops through each selected book and decreases available_copies by one. 6. The transaction email workflow sends its notification, and the transaction follows the Book Issue Process Blueprint.
Alternate Flow	If a selected book has no available copies, the Block Issuance If No Copies workflow restricts the issue. If a book has available_copies equal to zero, the stock is never reduced below zero.
Post-condition	The transaction record exists in All Book Transactions with status Issued, and the Available Copies of each selected book are reduced by one.
Related FR	FR-05, FR-06, FR-07, FR-12, FR-14

Table 3.6: Use Case Description – Return Book (UC-05)

Use Case ID	UC-05
Use Case Name	Return Book
Actor(s)	Librarian (primary); Administrator
Pre-condition	An Issued transaction exists in Book Transactions and the actor’s role allows editing of the Book Transaction form.
Main Flow	1. The actor opens All Book Transactions and opens the Issued transaction. 2. The actor sets Status to Returned and enters the Return Date. 3. The actor saves the record. 4. The return workflow runs for the record whose Return Processed box is unchecked, increases available_copies by one for each selected book and sets return_processed to true. 5. Process Book Return uses Calculate_Fine(due_date, return_date) to find any overdue fine at ₹50 per overdue day and stores it in Fine Amount.
Alternate Flow	If the same returned transaction is edited and saved again, return_processed is already true, so the stock is not increased a second time. If the book is not overdue, no fine is charged. A fine can then be recorded in the Fine Payment form.
Post-condition	The transaction shows status Returned, the stock is restored once (verified from 10 to 11 copies in testing), and any fine is available for payment recording.
Related FR	FR-08, FR-09, FR-11, FR-14

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The application runs entirely inside Zoho Creator; it has no custom front end, separate back end or external database. Figure 4.1 shows the architecture. Administrators, Librarians and Students use the application through a web browser. The presentation layer consists of the four forms (Book Details, Student Details, Book Transaction and Fine Payment), the reports with their category and department views, and the Dashboard page. The logic layer consists of Deluge form workflows (Fetch_Book_Details_API, Auto-Update Available Copies, Block Issuance If No Copies and the return processing workflow), the Calculate_Fine function used by Process Book Return, two scheduled workflows (Bulk_Overdue_Fine_Update and the daily overdue reminder), the transaction email workflow and the Book Issue Process Blueprint. The data layer is the set of Book Details, Student Details, Book Transaction and Fine Payment records held by Creator and related through lookup fields, together with the Additional Authors subform of Book Details. Fetch_Book_Details_API calls the Open Library service through Deluge invokeurl, and the e-mail workflows send messages through the e-mail service. Access is governed by the Admin, Librarian and Student roles and their permission settings.

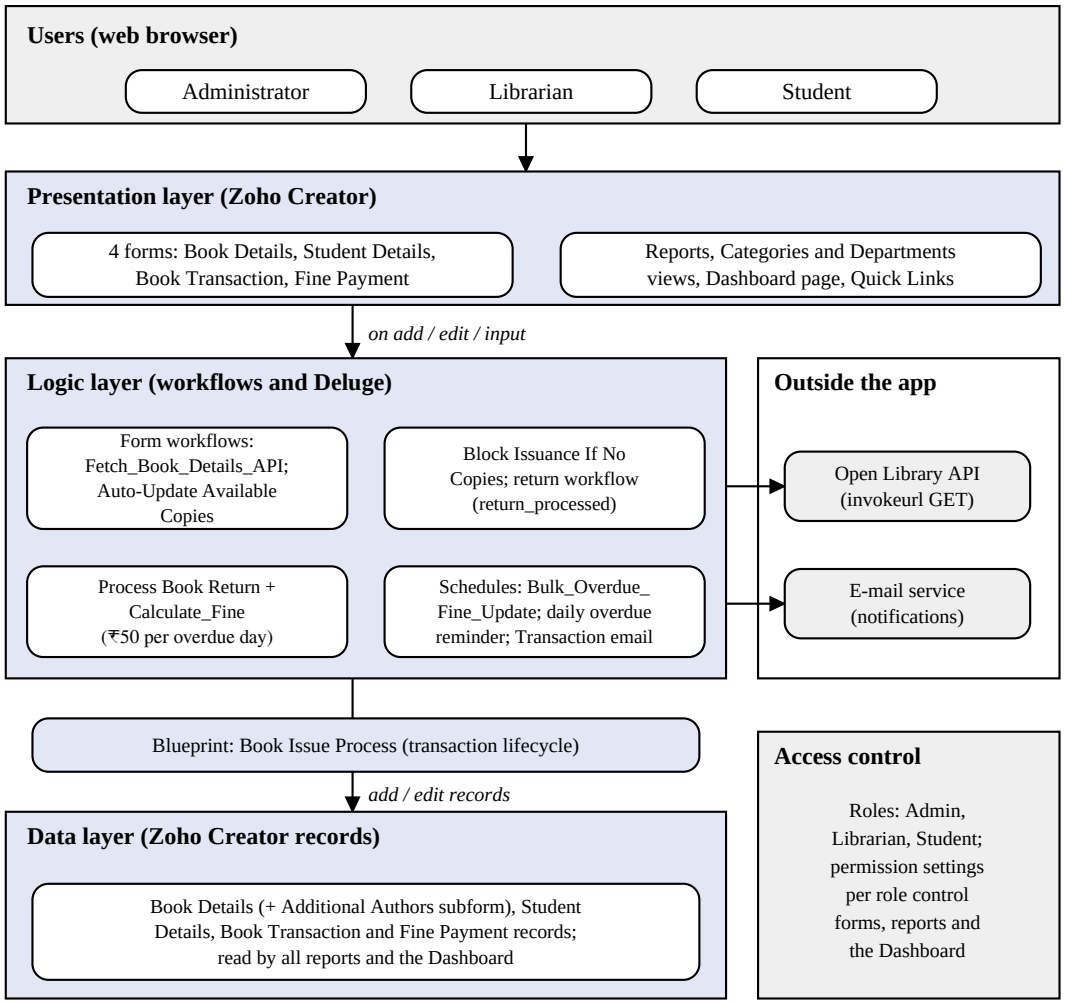


Figure 4.1: System Architecture of the Smart Library Management System

4.2 Data Model

Zoho Creator stores data as records of forms. A form therefore plays the role of a table in a relational database, a field plays the role of a column, and a lookup field plays the role of a foreign key that points to a record of another form. The application uses four forms. Book Details holds the catalogue record of each book, including its stock counts. Student Details holds the library member and is the source of the Student lookup. Book Transaction holds each borrowing transaction and carries a Book lookup, in which Multi Select is enabled so that one transaction can contain several books, and a Student lookup. Fine Payment holds the payment details and carries a Transaction lookup to Book Transaction. Book Details also contains the Additional Authors subform, which keeps the extra author entries of a book inside the book record because they have no use outside it. Figure 4.2 shows the entities and relationships and Table 4.1 summarises them. Cardinalities are stated only as far as the Creator configuration records them.

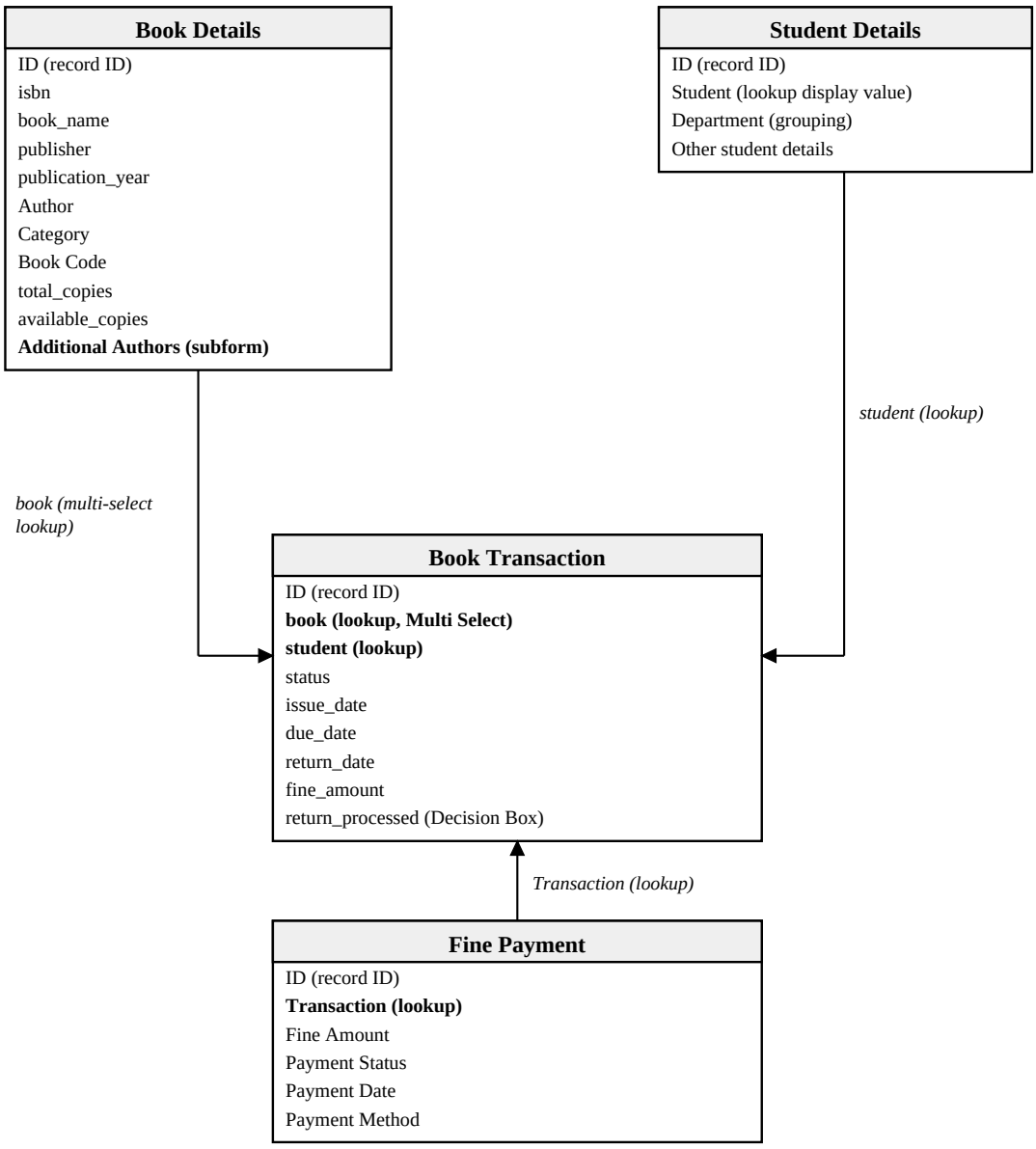


Figure 4.2: Entity–Relationship Diagram of the Application

Data consistency is supported in five ways. First, books and students are chosen from existing records through lookups, so a transaction cannot name a book or a student that does not exist. Second, the ISBN autofill copies the title, publisher and publication year from Open Library, which reduces typing errors in the catalogue. Third, available_copies is changed only by workflows: it is decreased once for each selected book when an Issued transaction is added, never below zero, and increased once for each selected book on return. Fourth, the Return Processed decision box (return_processed, default unchecked) ensures that a returned transaction restores stock only once, even if the record is edited and saved again. Fifth, the fine is always recalculated as the total of overdue days × ₹50, so it is not added repeatedly by each

scheduled run.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Book Details	isbn, book_name, publisher, publication_year, total_copies, available_copies	Book Transaction; Additional Authors subform	Referenced by transactions (a transaction can hold several books); one book has its own additional authors	Book lookup (Multi Select) in Book Transaction; Additional Authors subform
Student Details	Student record and department details	Book Transaction	Referenced by transactions as the borrower	Student lookup in Book Transaction
Book Transaction	book, student, status, issue_date, due_date, return_date, fine_amount, return_processed	Book Details; Student Details; Fine Payment	Lookup to Book Details (multi-select); lookup to Student Details; referenced by Fine Payment	book and student lookups; Transaction lookup in Fine Payment
Fine Payment	Transaction, Fine Amount, Payment Status, Payment Date, Payment Method	Book Transaction	Lookup to the transaction whose fine is paid	Transaction lookup

4.3 Form Design

Tables 4.2 to 4.5 specify the four forms. The field link names written without an asterisk are the names used in the Deluge scripts of this project (isbn, book_name, publisher, publication_year, available_copies, total_copies, book, student, status, issue_date, due_date, return_date, fine_amount and return_processed). A name marked with an asterisk follows Creator’s default convention of using lower-case letters and replacing spaces with underscores, and should be confirmed in the form builder shown in Chapter 5. The column Data Held describes what a field stores, and the column Set By shows whether the value is typed or selected by the user, filled by the ISBN service, or written by a workflow. The complete builder settings of each form appear in the screenshots of Chapter 5.

Table 4.2: Field Specification – Book Details Form

Field Label	Link Name	Data Held	Set By	Validation / Default	Remarks
ISBN	isbn	ISBN identifier	User entry	On User Input event starts the ISBN workflow	Key for the API call
Book Name	book_name	Book title	ISBN service	Filled when the response contains it	Open Library title

Field Label	Link Name	Data Held	Set By	Validation / Default	Remarks
Publisher	publisher	Publisher name	ISBN service	Filled when the response contains it	Open Library publisher
Publication Year	publication_year	Year of publication	ISBN service	Filled when the response contains it	Open Library year
Author	author*	Author name	User entry	Not filled by the API	Entered manually
Additional Authors	Subform (name confirmed in builder)	Extra author entries	User entry	Subform inside Book Details	Rows added per extra author
Category	category*	Book category	User entry	—	Basis of the Categories view
Book Code	book_code*	Book identification code	User entry	—	Used when selecting books
Total Copies	total_copies	Total stock of the book	User entry	—	Total quantity held
Available Copies	available_copies	Copies free for issue	Workflows	Decreased on issue (never below 0); increased on return	Stock rule of the system

Table 4.3: Design Elements – Student Details Form

Element	Role in the Design	Used By
Student record	One record per library member (borrower); the record is chosen through a lookup.	student lookup of Book Transaction
Department	Used to group and list students in the Departments entry of the menu.	Departments report
Student details	Member information stored once per student and shown in All Student Details; the exact field inventory is given by the form builder screenshot in Chapter 5.	All Student Details report; role-based access

Table 4.4: Field Specification – Book Transaction Form

Field Label	Link Name	Data Held	Set By	Validation / Default	Remarks
Book	book	Lookup to Book Details; Multi Select enabled	Selection	Deluge type bigintlist; scripts loop over every selected ID	One transaction, several books
Student	student	Lookup to Student Details	Selection	—	Identifies the borrower
Status	status	Transaction status	Selection	Issued and Returned drive the workflows	Separates issue and return

Field Label	Link Name	Data Held	Set By	Validation / Default	Remarks
Issue Date	issue_date	Date of issue	User entry	—	—
Due Date	due_date	Date to return the book	User entry	Must be non-null for the scheduled fine update	Basis of overdue days
Return Date	return_date	Date the book is returned	User entry	—	Input of Calculate_Fine
Fine Amount	fine_amount	Calculated fine in ₹	Workflows	₹50 × overdue days	Written by the fine logic
Return Processed	return_processed	Decision Box	Workflow	Default: unchecked	Duplicate-return guard

Table 4.5: Field Specification – Fine Payment Form

Field Label	Link Name	Data Held	Set By	Validation / Default	Remarks
Transaction	transaction*	Lookup to Book Transaction	Selection	—	Links payment to the loan
Fine Amount	fine_amount*	Amount of the fine	User entry	—	Amount associated with payment
Payment Status	payment_status*	Status of payment	Selection	—	Shows paid or unpaid state
Payment Date	payment_date*	Date of payment	User entry	—	—
Payment Method	payment_method*	Method used to pay	Selection	—	—
Payment Reference	payment_reference*	Reference of payment	User entry	Optional, if present in the form	Optional field
Remarks	remarks*	Free-text note	User entry	Optional, if present in the form	Optional field

4.4 Process and Workflow Design

Workflow inventory. Table 4.6 lists the automation of the application. Each rule is placed in its own named workflow or function so that it can be maintained separately. During testing, duplicate workflows named Check_Book_Availability were found, several with no actions. The problematic duplicates were disabled and Block Issuance If No Copies was kept as the single active no-copies restriction, so the duplicates are not part of the design.

Table 4.6: Workflow and Function Inventory

ID	Workflow / Function	Form	Trigger	Purpose
WF-01	Fetch_Book_Details_API	Book Details	On User Input of ISBN	invokeurl GET to https://openlibrary.org/isbn/{ISBN}.json; fills book_name, publisher and publication_year
WF-02	Auto-Update Available Copies	Book Transaction	On Add → On Success; status == "Issued" and book != null	Decreases available_copies by one for each selected book if the count is above zero
WF-03	Block Issuance If No Copies	Book Transaction	Form workflow on issue	Restricts the issue of a book when no copies are available
WF-04	Return processing workflow	Book Transaction	Status is Returned and Return Processed is false	Increases available_copies by one for each selected book and sets return_processed to true
WF-05	Calculate_Fine(due_date, return_date) with Process Book Return	Book Transaction	Return processing	Calculates the fine at ₹50 per overdue day
WF-06	Bulk_Overdue_Fine_Update	Book Transaction	On Schedule; start 02-Nov-2026 10:00:00; batch size 1000; enabled	For Issued records with a due date, sets fine_amount = overdueDays × 50 when overdueDays > 0
WF-07	Transaction email workflow	Book Transaction	Transaction event	Sends the configured transaction e-mail
WF-08	Daily overdue reminder	Book Transaction	Scheduled daily	Identifies overdue transactions and sends the configured reminder
WF-09	Book Issue Process (Blueprint)	Book Transaction	Transaction lifecycle	Guides the transaction through the stages of Table 4.7

Book issue processing. Figure 4.3 shows what happens when a book is issued. The Librarian adds a Book Transaction with one or more books, the student, the status Issued, the issue date and the due date. If no copy is available, Block Issuance If No Copies restricts the issue. Otherwise the record is saved and, on success, Auto-Update Available Copies runs when the status is Issued and a book is selected. It loops through every selected book ID, fetches the Book Details record and decreases available_copies by one when the current count is greater than zero. The transaction email workflow then sends its notification and the transaction follows the Book Issue Process Blueprint.

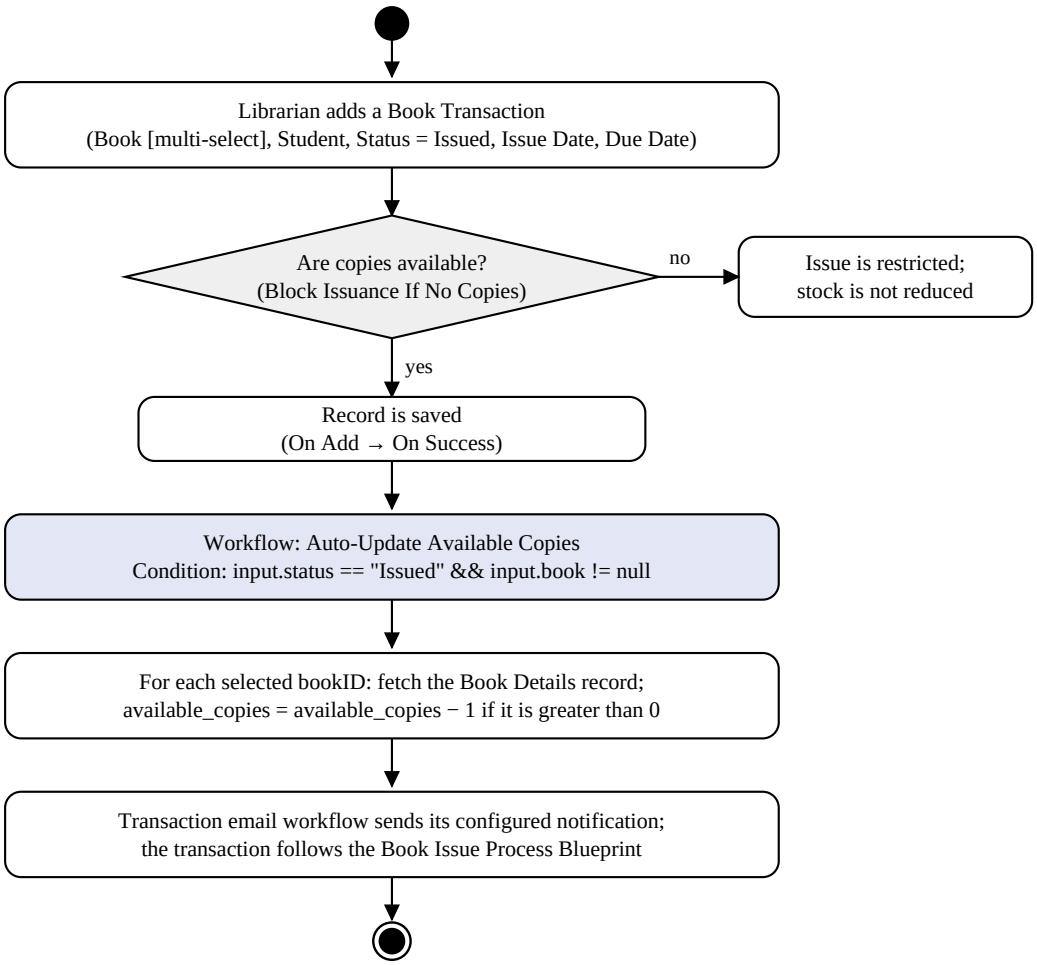


Figure 4.3: Workflow / Process Flow for Issuing a Book

Return and fine processing. Figure 4.4 shows the return of a book. When the status becomes Returned and Return Processed is unchecked, the return workflow increases available_copies by one for each selected book and then sets return_processed to true. If the same transaction is saved again, the check box is already ticked, so stock is not increased a second time. Process Book Return uses the function Calculate_Fine(due_date, return_date) to find the fine at ₹50 per overdue day, and the result is held in fine_amount. A payment can then be recorded in the Fine Payment form against the transaction.

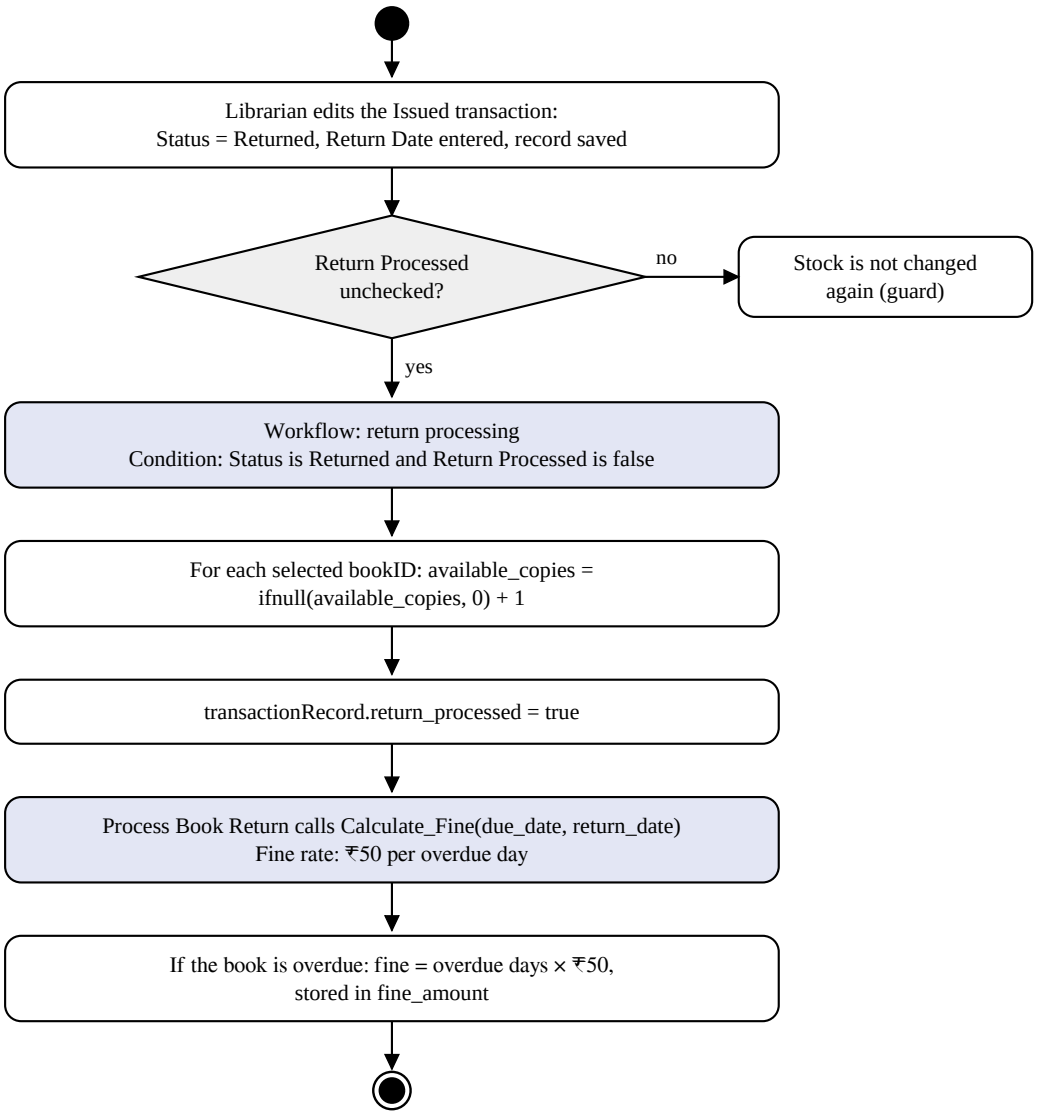


Figure 4.4: Workflow / Process Flow for Returning a Book and Calculating the Fine

Scheduled processing. Figure 4.5 shows the two scheduled workflows. Bulk_Overdue_Fine_Update runs on a schedule from 02-Nov-2026 at 10:00:00 with a batch size of 1000. For each Issued transaction that has a due date it computes overdueDays with daysBetween(due_date, zoho.currentdate) and, when the value is above zero, sets fine_amount to overdueDays × 50. The fine is therefore a total that is recomputed on each run and not a running sum. The script does not reset fine_amount when overdueDays is zero. The daily overdue reminder runs every day, identifies overdue transactions and sends the configured reminder.

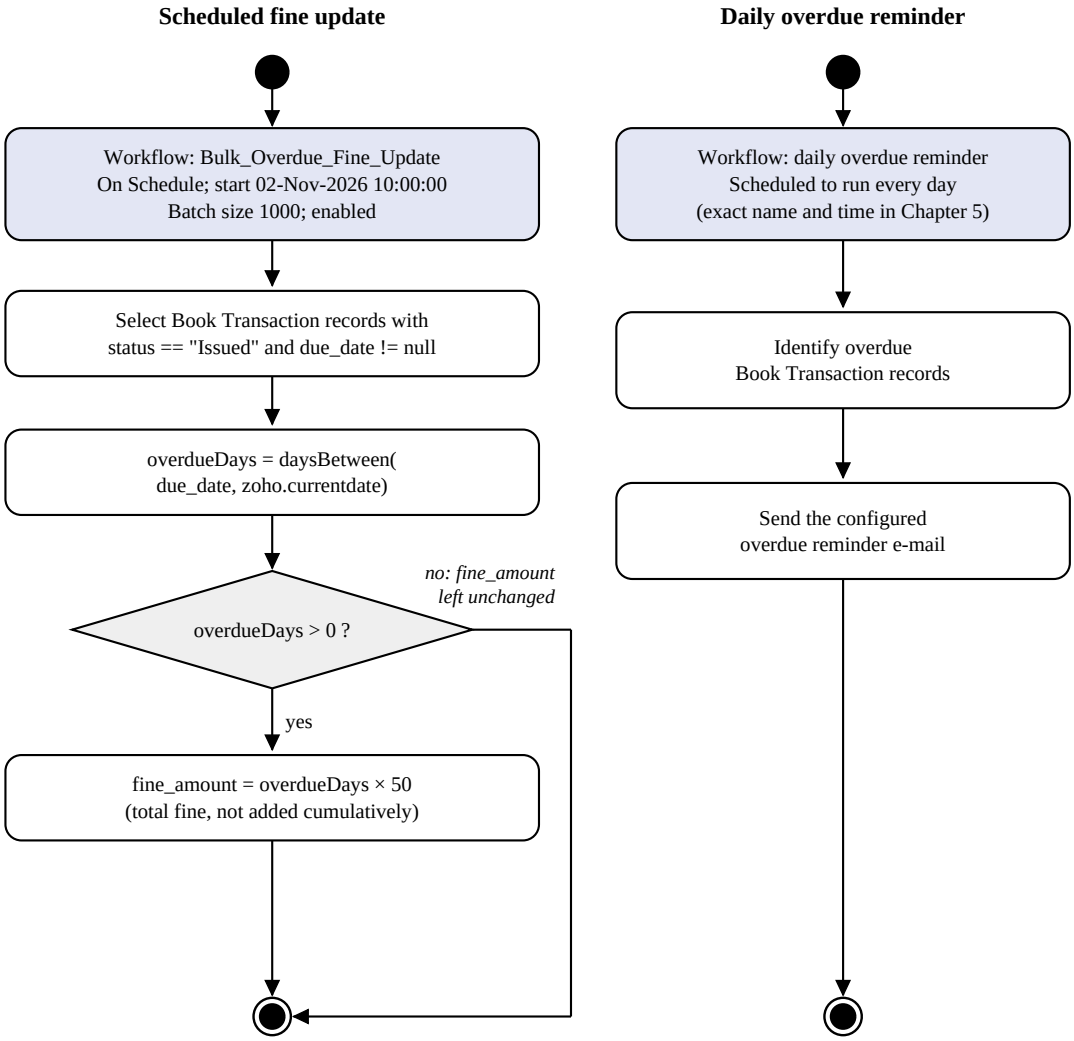


Figure 4.5: Scheduled Fine Update and Daily Overdue Reminder Flows

ISBN autofill. When the user enters an ISBN, the On User Input event of Fetch_Book_Details_API builds the Open Library URL, makes an HTTP GET request with invokeurl, parses the JSON response and assigns the values to book_name, publisher and publication_year. The author is not supplied by this script and is entered manually. In the test, ISBN 9783161484100 returned Logik der Forschung, Mohr Siebeck and 2005. This is an external REST API call; it is not a webhook and not a Zoho Creator REST API endpoint.

Transaction lifecycle (Blueprint). The Book Issue Process Blueprint represents the lifecycle of a library transaction. Figure 4.6 and Table 4.7 show its thirteen stages. The main column follows the order of the stages in the Blueprint diagram, while Transaction Rejected, Transaction Cancelled, Lost Book Reported and Replacement Ordered are exit or exception stages. The transitions observed in the Blueprint are Submit for Approval, Approve Transaction, Cancel Request, Reject Transaction,

Confirm Issue, Void Issue, Initiate Return, Mark Overdue, Report Book Lost, Cancel Return, Process Return, Apply Fine, Close Transaction, Process Payment, Return Book, Assess Replacement Cost and Order Replacement. They are the actions that move a transaction from one stage to another. The Blueprint was saved, enabled and tested, and its stage-level criteria are shown in the screenshots of Chapter 5.

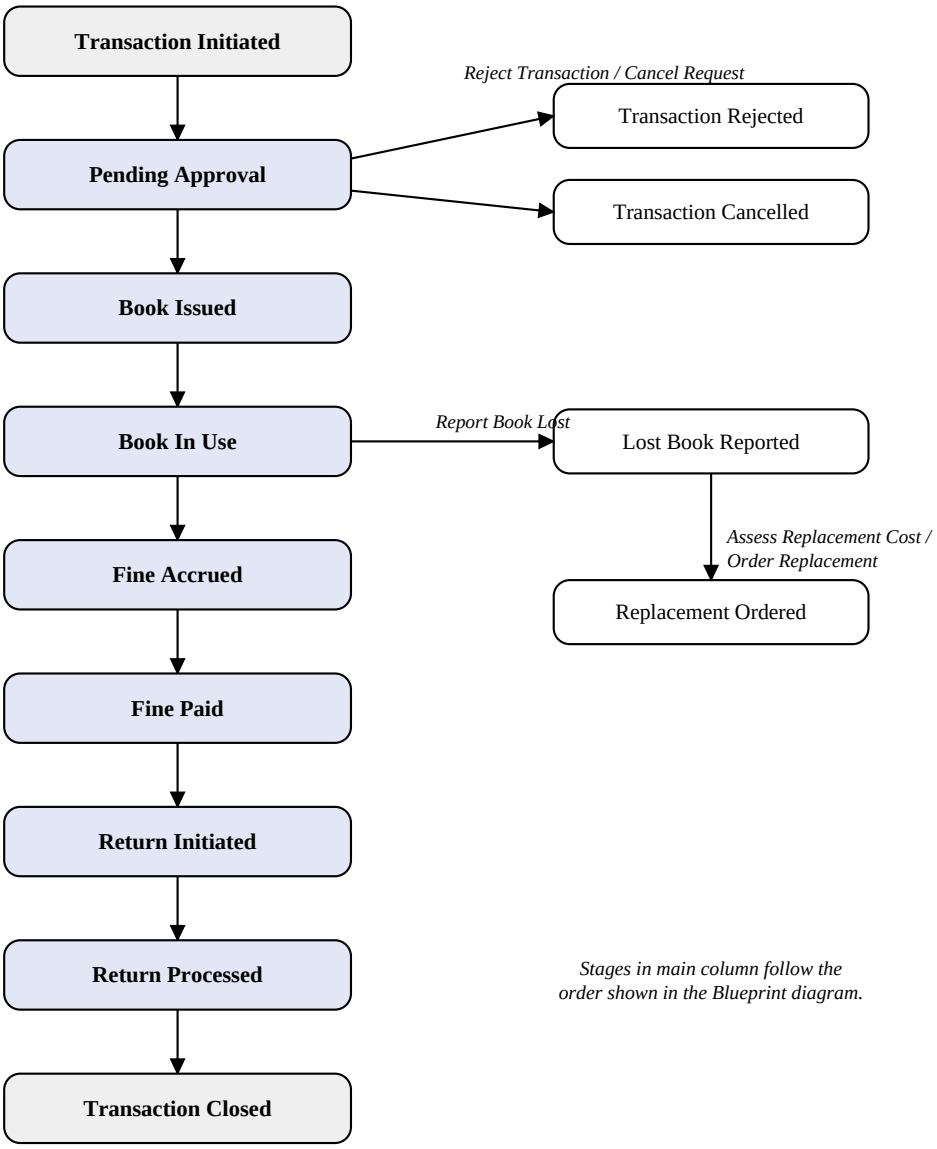


Figure 4.6: Stages of the Book Issue Process Blueprint

Table 4.7: Stages of the Book Issue Process Blueprint

Stage	Role in the Lifecycle	Path
Transaction Initiated	A new borrowing transaction enters the process.	Main
Pending Approval	The transaction awaits approval where the process requires it.	Main
Book Issued	The issue is confirmed.	Main
Book In Use	The borrowed book remains with the student.	Main
Fine Accrued	An overdue fine is applied or recorded.	Main
Fine Paid	The fine payment is completed.	Main
Return Initiated	The student or the library starts the return process.	Main
Return Processed	The return operation has been completed.	Main
Transaction Closed	The transaction lifecycle is completed.	Main
Transaction Cancelled	The transaction is cancelled.	Exit
Transaction Rejected	The request is rejected.	Exit
Lost Book Reported	The book is marked as lost.	Exception
Replacement Ordered	The replacement process is started.	Exception

4.5 User Interface and Navigation Design

Figure 4.7 shows the navigation structure of the application menu. The menu has the sections Book Details, Student Details, Book Transactions and Fine Payments, plus the Dashboard page. Book Details holds the Book Details entry, the Book Details Report and Categories; Student Details holds Student Details, All Student Details and Departments; Book Transactions holds Book Transactions and All Book Transactions; and Fine Payments holds Fine Payments and All Fine Payments. The Dashboard combines summary cards, charts, year-wise statistics and Quick Links on one page. The dashboard components were retained as built, including some duplicated or awkward labels such as the repeated Total Fine Amounts and the label Total Total Copies, and are documented in Chapter 5 as they appear. The interface uses the platform’s standard side menu and form layout, as seen in the screenshots of Chapter 5.

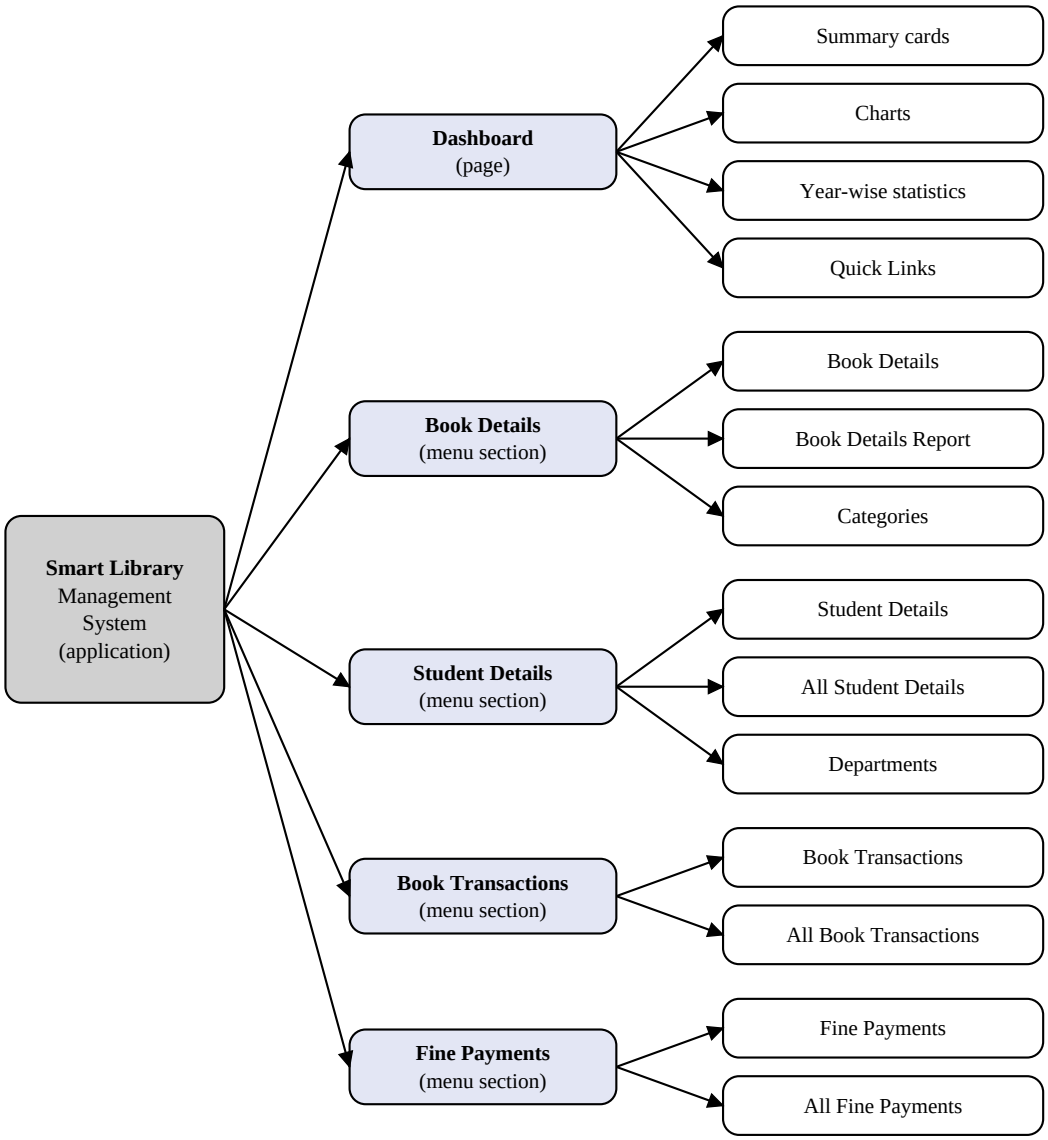


Figure 4.7: Navigation Structure of the Application

4.6 Access Control Design

Access is designed on three levels. Application access is by Zoho sign-in. Each user is then assigned one of three roles: Admin, Librarian or Student. Permission settings define what each role may do with each form, report and page, so that responsibilities are separated and each role receives only the access it needs (least privilege). The Admin administers the application and library data, the Librarian performs library operations, and the Student uses only the permitted student-facing functions with no administrative configuration. Table 4.8 records the design intent of the role–permission matrix, where Full = add, edit, view, delete and configuration; Manage = day-to-day operation of the records (add, edit and view as permitted); View = view of permitted records only; and — = no access. The exact permission profile names and toggles configured in Zoho Creator are reproduced from the permission screens in Chapter 5, and no passwords or credentials are recorded in this report.

Table 4.8: Role–Permission Matrix (Design Intent)

Component	Admin	Librarian	Student
Form: Book Details	Full	Manage	View (available books)
Form: Student Details	Full	Manage (as permitted)	—
Form: Book Transaction	Full	Manage (issue and return)	View (own borrowing history)
Form: Fine Payment	Full	Manage	—
Reports (books, students, transactions, fines)	Full	Manage	View (permitted reports)
Page: Dashboard	Full	View	—
Workflows, Blueprint and application settings	Full	—	—
Roles and permission settings	Full	—	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in a Zoho Creator account (India data centre) for the Smart Library Management System project. Inside Creator the application carries the name “College Library Management System”, which is the name shown in the application header in every screenshot of this chapter. New forms were added with the Create from Scratch option, which asks only for a form name (Figure 5.1). Four forms were created: Book Details, Student Details, Book Transaction and Fine Payment.

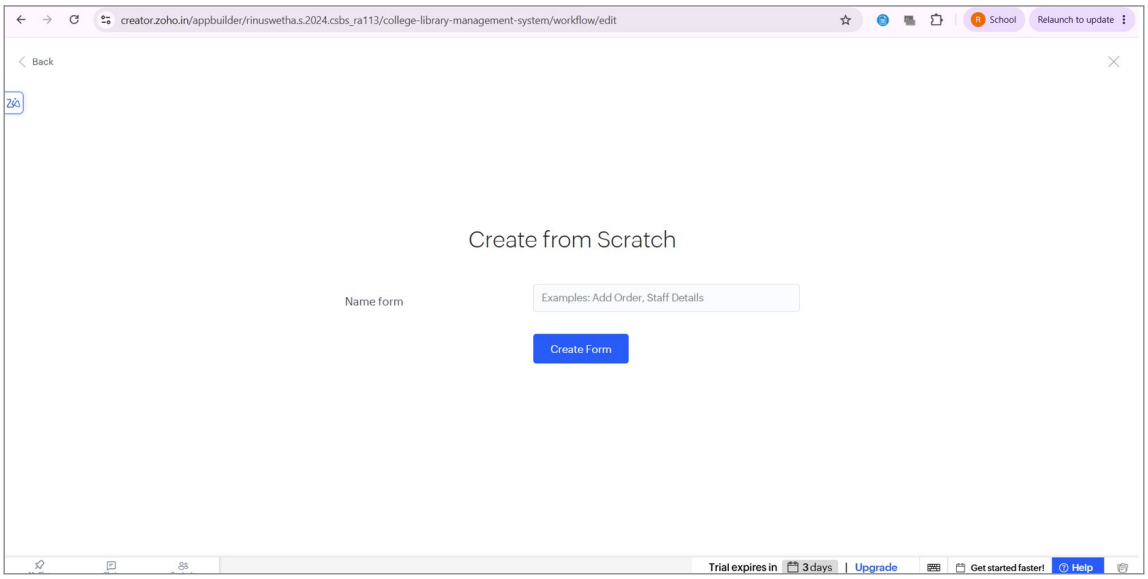


Figure 5.1: Creating a Form with the Create from Scratch Option in the Creator Builder

The application menu was organised into the sections Book Transactions, Fine Payments, Book Details and Student Details, with the Dashboard page as the first entry and an Approvals section at the end (Figure 5.2). Each data section holds its data-entry form together with its reports: Book Transactions contains Book Transaction and All Book Transactions; Fine Payments contains Fine Payment and All Fine Payments; Book Details contains Book Details, Book Details Report and Categories; and Student Details contains Student Details, All Student Details and Departments.

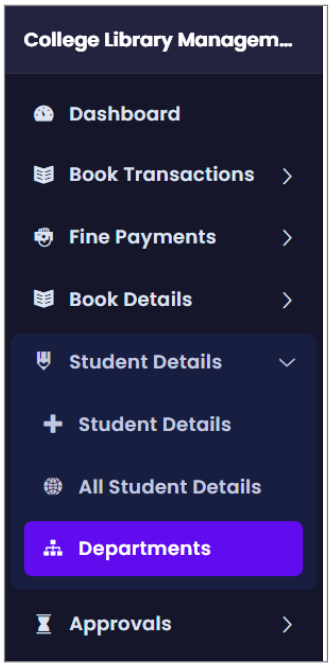


Figure 5.2: Application Menu with the Student Details Section Expanded

The application header shows that the account was on a trial edition, with 3 days remaining, when the screenshots were taken. The application link name is college-library-management-system and the application address is https://creatorapp.zoho.in/rinuswetha.s.2024.csbs_ra113/college-library-management-system.

5.2 Forms and Data Capture

Book Details form. Figure 5.3 shows the Book Details form in the builder. It holds Book Name, Author, ISBN, Category, Publisher, Total Copies, Available Copies, Additional Authors, Book Code and Publication Year. Category is a drop-down, Total Copies, Available Copies and Publication Year are number fields, and the remaining fields are single-line text fields. The properties panel for Book Name shows the link name book_name and a character maximum of 255; the options Mandatory, No duplicate values and “Contains personal data (PII)” are not ticked. The ISBN field is the field that starts the Open Library lookup (Section 5.7), and Available Copies is the field that the stock workflows update (Section 5.6).

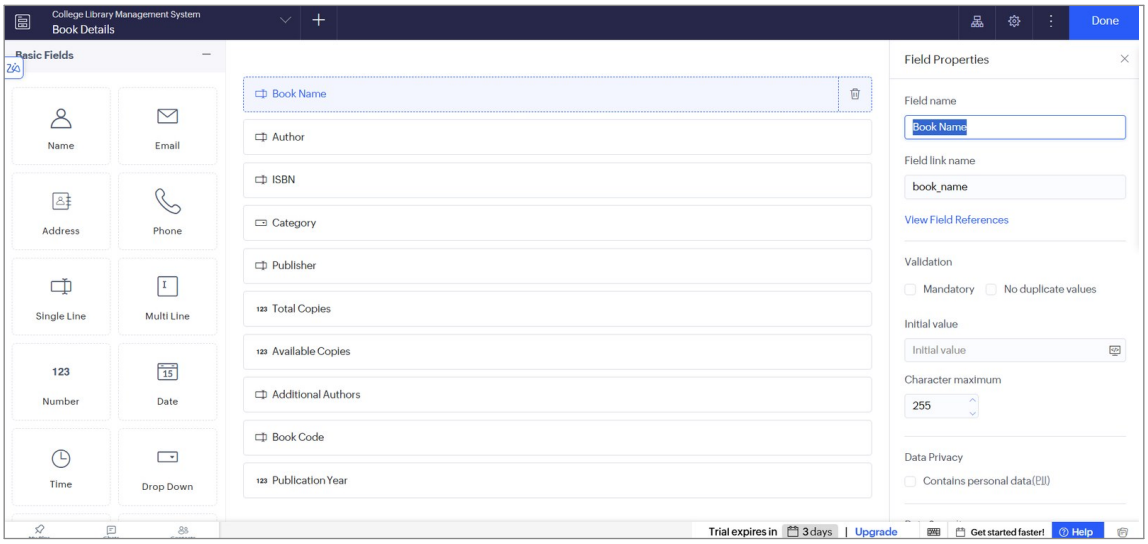


Figure 5.3: Book Details Form in the Form Builder

Student Details form. Figure 5.4 shows the Student Details form. Student Name is a Name field with only the First Name display field ticked (Prefix, Last Name and Suffix are not ticked); its link name is student_name and Mandatory is not ticked. Register Number and Student Code are single-line fields, Email is an e-mail field, Department and Year are drop-downs, Phone Number and Emergency Contact are phone fields, and Address is an address field. The Year choices seen in the application are First, Second, Third and Fourth.

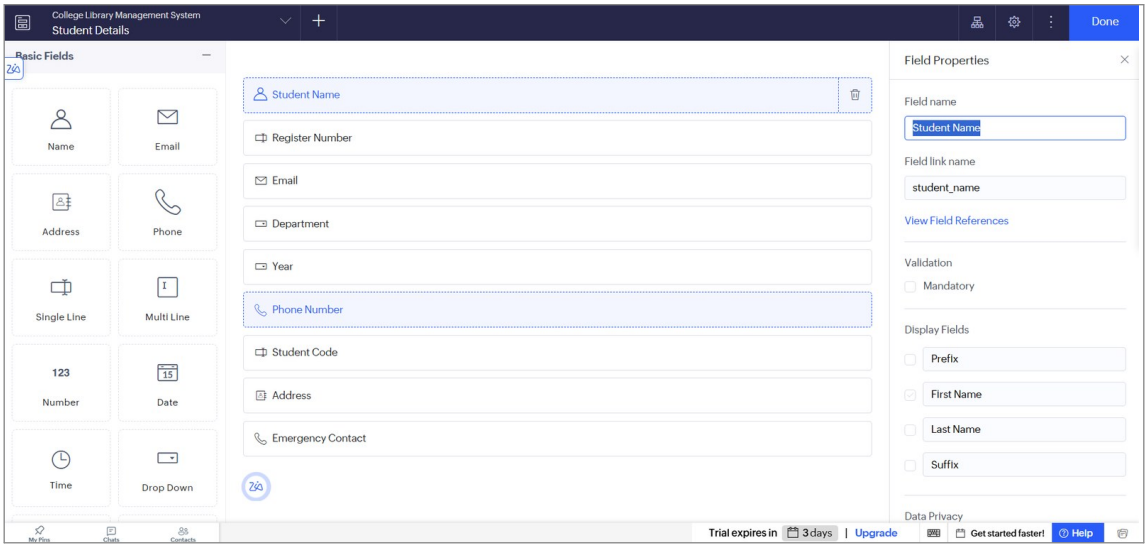


Figure 5.4: Student Details Form in the Form Builder

Book Transaction form. Figure 5.5 shows the Book Transaction form. Transaction ID is a single-line field with the link name transaction_id (Mandatory and No duplicate values are not ticked, character maximum 255). Student and Book are lookup fields, Issue Date, Due Date and Return Date are date fields, Status is a radio field, Fine Amount is a currency field, Transaction Reference is a single-line field and Return Processed is a decision box. The Book field is a

multi-select lookup to Book Details, so one transaction can include more than one book. Return Processed is unticked by default and is used by the return workflow (Section 5.6).

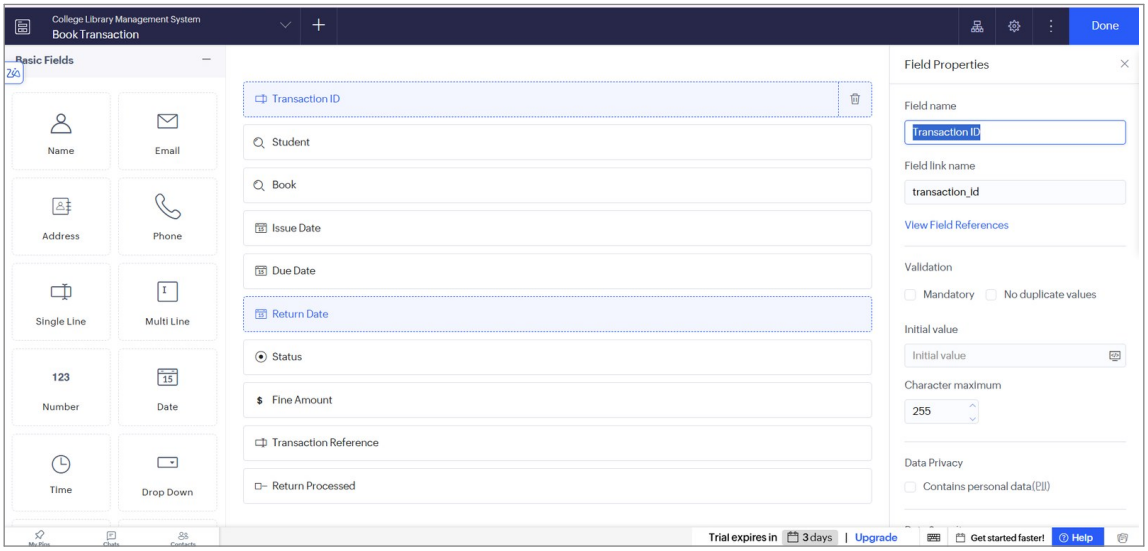


Figure 5.5: Book Transaction Form in the Form Builder

Fine Payment form. Figure 5.6 shows the Fine Payment form with the fields Fine ID (link name fine_id), Transaction (lookup to Book Transaction), Fine Amount (currency), Payment Status (radio), Payment Date (date), Payment Method (radio), Payment Reference (single line) and Remarks (multi line). The choices of Payment Status are Paid, Pending and Other, and the choices of Payment Method are Cash, Online and Other (Figure 5.10).

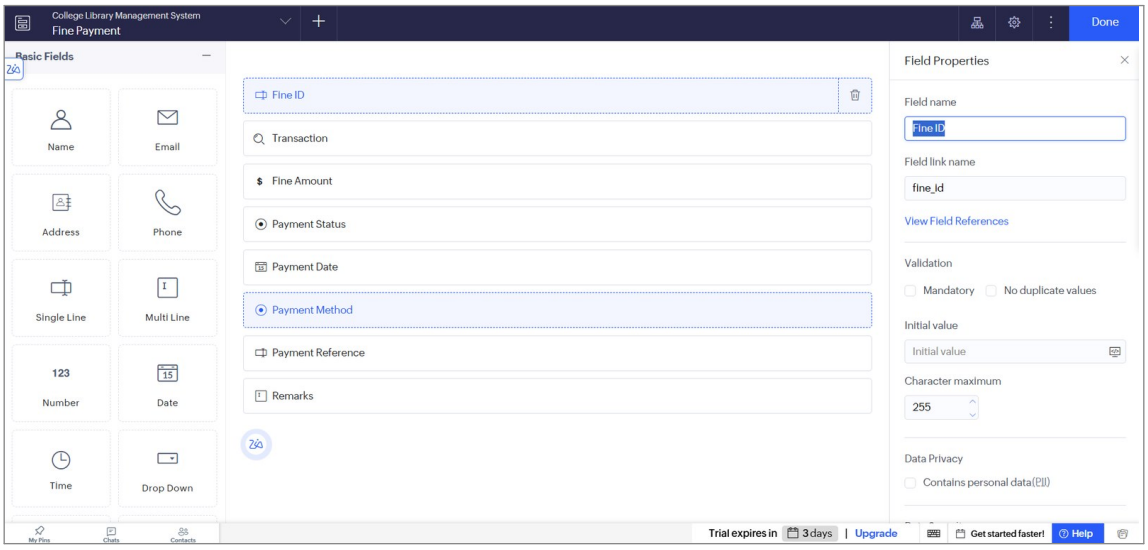


Figure 5.6: Fine Payment Form in the Form Builder

Live mode. Figures 5.7 to 5.10 show the four forms in live mode. Dates are entered in the format dd-MMM-yyyy, the Fine Amount fields show a numeric entry mask, and the phone fields of the Student Details form carry the India country code +91 with a sample number as placeholder. Category, Department and Year are chosen from drop-downs. On the Book

Transaction form the Student and Book fields are chosen from lookups (“-Select-”) and Status is chosen from Returned, Issued and Other (Figure 5.9). The Book Details and Student Details forms end with the Submit and Reset buttons.

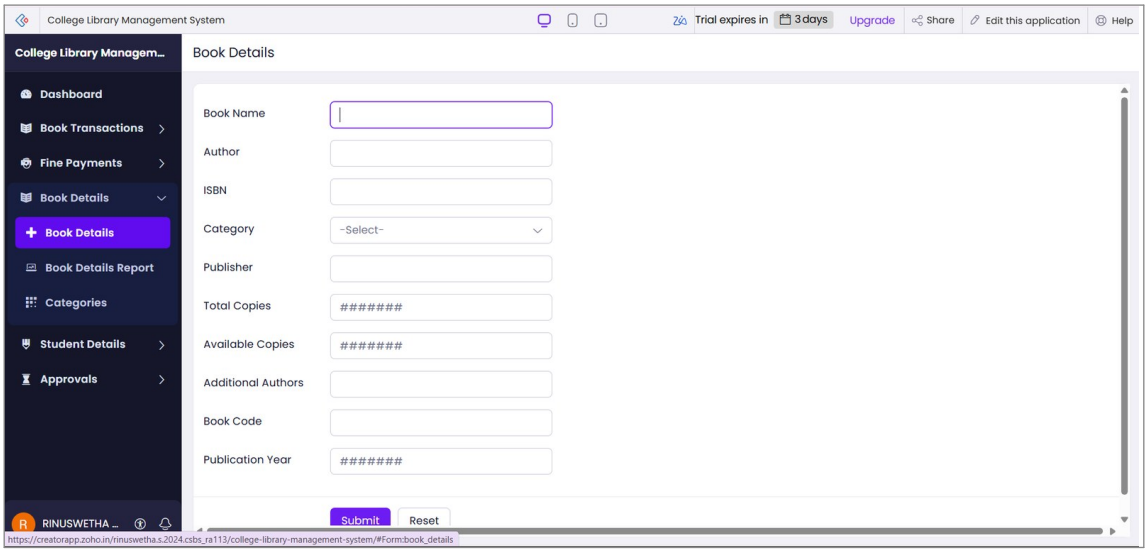


Figure 5.7: Book Details Form in Live Mode

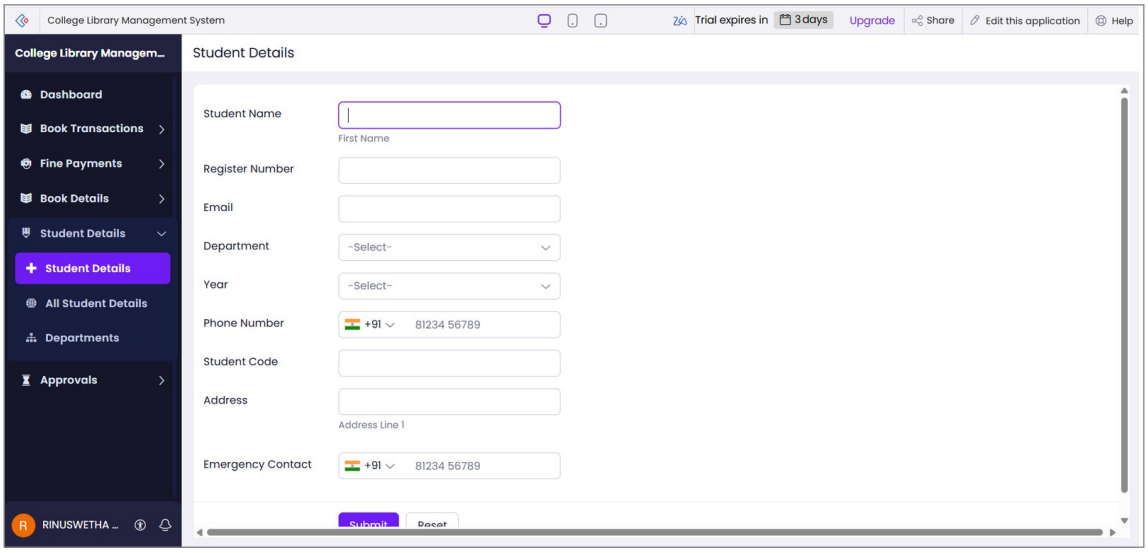


Figure 5.8: Student Details Form in Live Mode

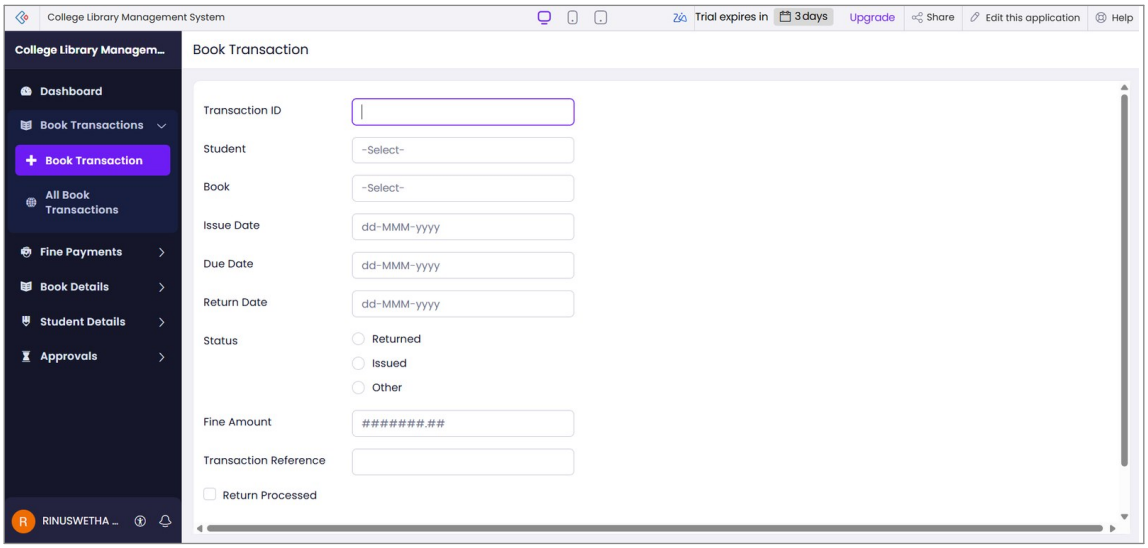


Figure 5.9: Book Transaction Form in Live Mode

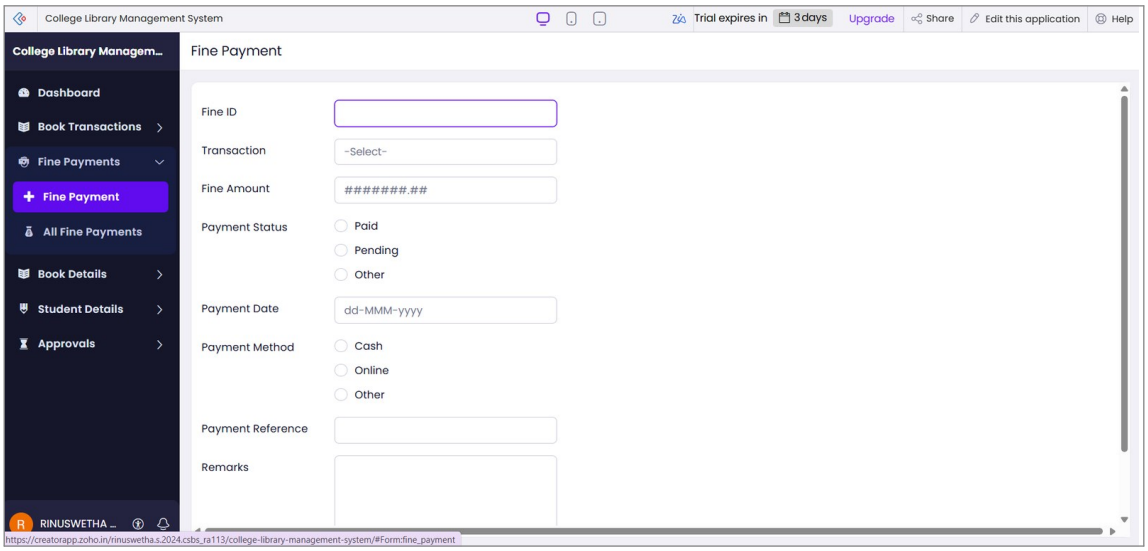


Figure 5.10: Fine Payment Form in Live Mode

5.3 Reports

A report was configured for each form and added to the menu of its section; Table 5.1 lists them. The four list reports show the records in rows, with a search icon, an add (+) button and a more-options (…) menu in the toolbar and a record count in the footer, for example “Showing 8 of 8”. Two Kanban reports arrange records in columns according to a field: Categories groups the Book Details records by Category and Departments groups the Student Details records by Department. The address bar of Figures 5.14, 5.15 and 5.16 shows the report link names book_details_by_category, student_details_Report and student_details_by_department.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Grouping / Display	Records Shown
All Book Transactions	List	Book Transaction	Transaction ID, Student, Book, dates, Status, Fine Amount, Transaction Reference	8 of 8
All Fine Payments	List	Fine Payment	Fine ID, Transaction, Fine Amount, Payment Status, Payment Date, Payment Method, Payment Reference, Remarks	3 of 3
Book Details Report	List	Book Details	Book Name, Author, ISBN, Category, Publisher, Total Copies, Available Copies and further columns	7 of 7
Categories	Kanban	Book Details	Columns by Category; cards show book name and author	Technology, Science, Non-Fiction, History and further columns
All Student Details	List	Student Details	Student Name, Register Number, Email, Department, Year (colour-coded), Phone Number, Student Code, Address and further columns	5 of 5
Departments	Kanban	Student Details	Columns by Department; cards show student name and register number	English, Chemistry, Computer Science, Electronics and further columns

All Book Transactions. Figure 5.11 shows eight transactions, five with the Transaction IDs T001 to T005 and three without a Transaction ID. The Student and Book columns display the Student Code and Book Code of the linked records (for example CODE1001 and BC-001), and some rows list several codes because more than one record is linked, as in T003. Fine Amount is shown in rupees. The record issued on 27-Oct-2026 with the due date 29-Oct-2026 and returned on 31-Oct-2026 shows a fine of ₹100.00, which is two overdue days at ₹50 per day (Section 5.6).

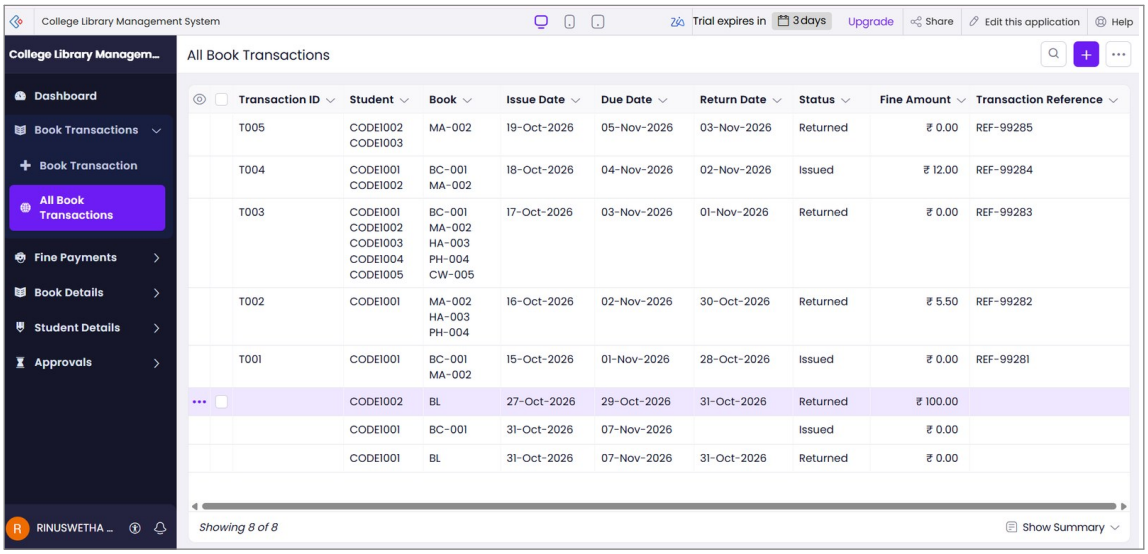


Figure 5.11: All Book Transactions Report (8 of 8 records)

All Fine Payments. Figure 5.12 shows three fine payment records. The Transaction column displays the Transaction Reference values (REF-99281 to REF-99285) of the linked transactions. The records have the Payment Status Paid, Pending and Pending and the Payment Method Cash, Online and Cash. Fine ID, Fine Amount, Payment Date, Payment Reference and Remarks are blank in these sample records.

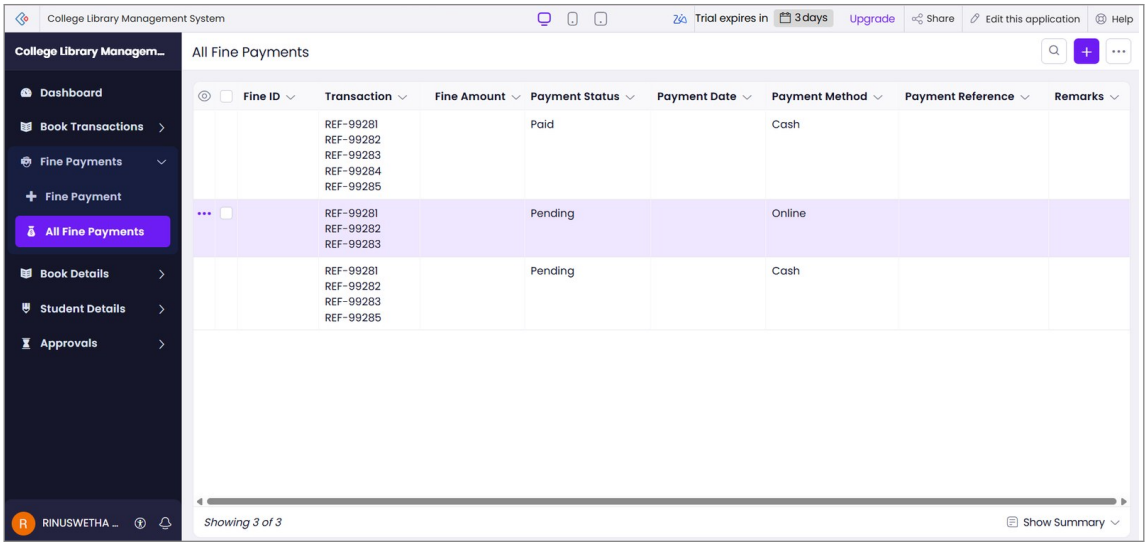


Figure 5.12: All Fine Payments Report (3 of 3 records)

Book Details Report. Figure 5.13 lists the seven books in the catalogue with their author, ISBN, category, publisher and copy counts. The catalogue includes Introduction to Python, Advanced Mathematics, History of Art, Modern Physics, Creative Writing Guide, Logik der Forschung (ISBN 9783161484100) and The C Programming Language (ISBN 9780131103627).

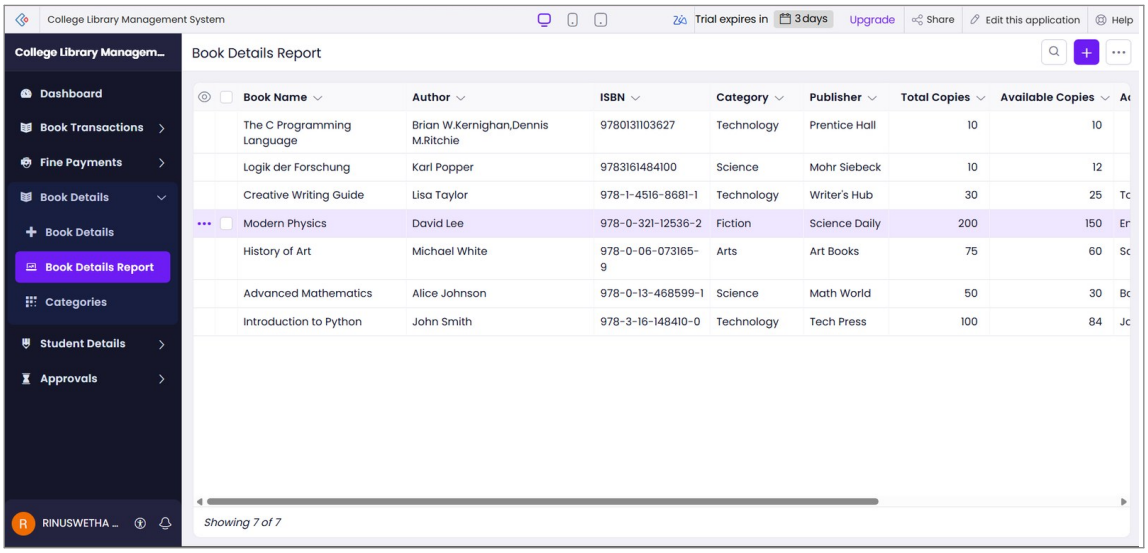


Figure 5.13: Book Details Report (7 of 7 records)

Categories. Figure 5.14 shows the Kanban report that groups the books by Category. The Technology column holds Introduction to Python, The C Programming Language and Creative Writing Guide; the Science column holds Logik der Forschung and Advanced Mathematics; the Non-Fiction column shows the message “No Records in Non-Fiction”; and further columns, such as History, lie to the right of the visible area.

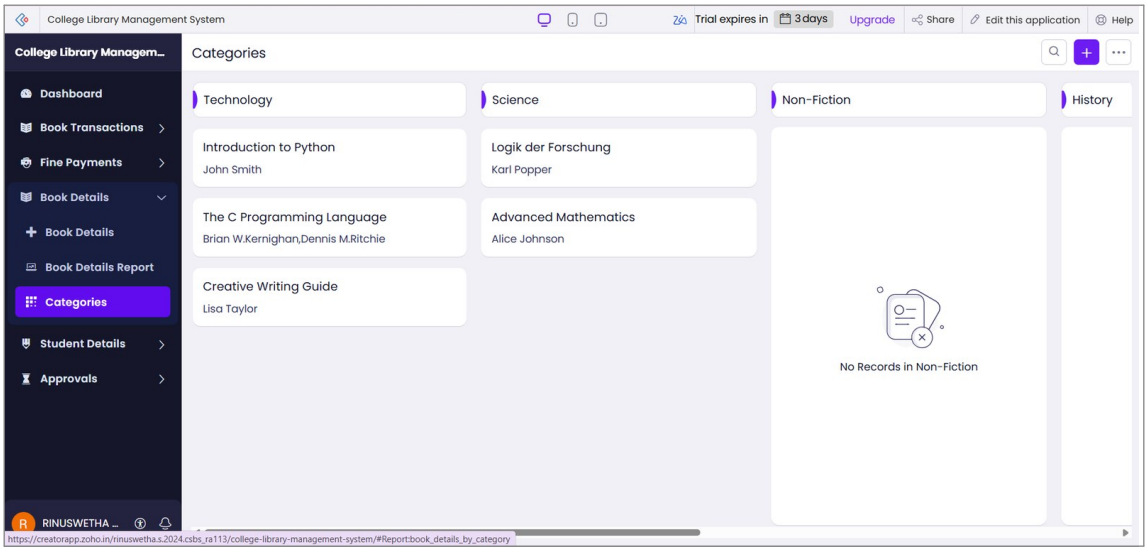


Figure 5.14: Categories Kanban Report

All Student Details. Figure 5.15 lists the five students. The Year column is colour-coded (Second in green, Third in red and Fourth in purple), the e-mail addresses are shown as links and the addresses carry a location marker.

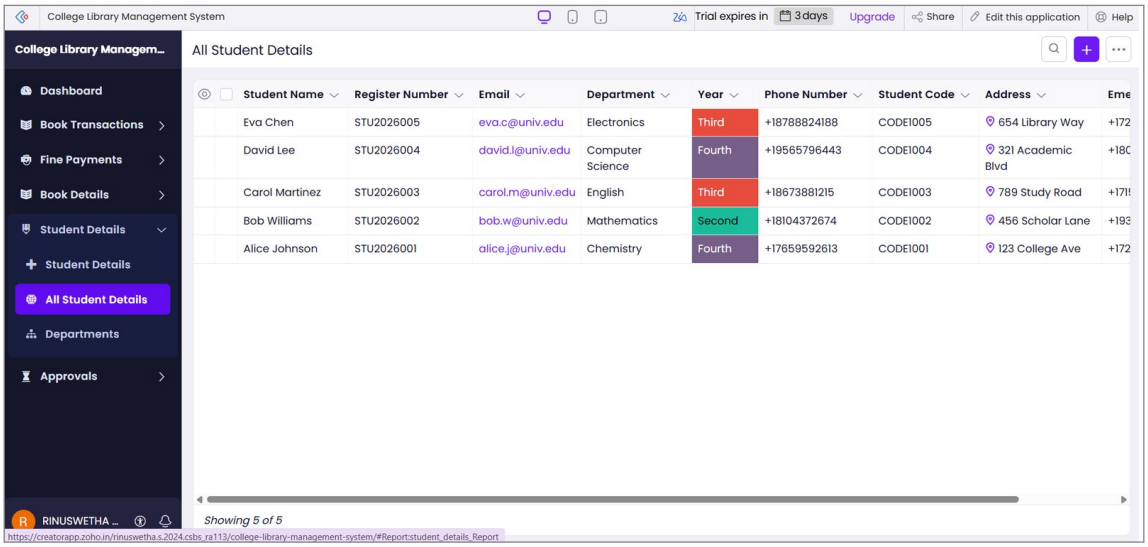


Figure 5.15: All Student Details Report (5 of 5 records)

Departments. Figure 5.16 shows the Kanban report that groups the students by Department. The English column holds Carol Martinez (STU2026003), Chemistry holds Alice Johnson (STU2026001), Computer Science holds David Lee (STU2026004) and Electronics holds Eva Chen; Mathematics lies further to the right.

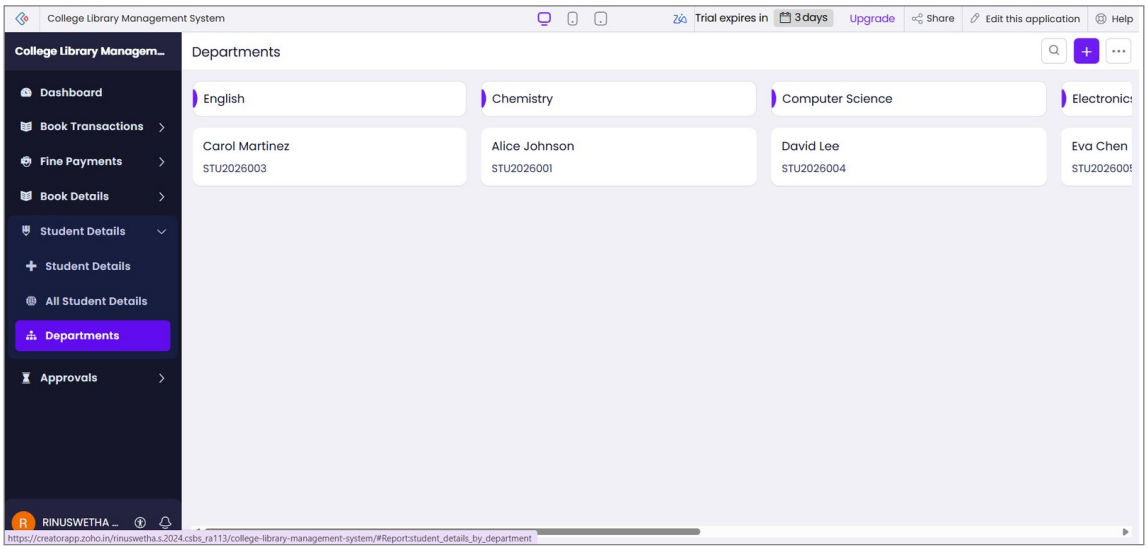


Figure 5.16: Departments Kanban Report

5.4 Pages and Dashboards

The page Dashboard is the first entry of the application menu (Figure 5.17). It contains summary cards, a year-wise count of students, two charts, a Quick Links panel and, below the charts, the All Book Transactions report (Figure 5.18). Table 5.2 lists the components. The dashboard is shown as built, and the card labels appear exactly as displayed in the application, including the label “Total Total Copies”.

Table 5.2: Dashboard Components

Component	Type	Content Displayed	Purpose
Total Fine Amounts	Summary card	₹ 117.50	Total of the fine amounts of all transactions
Total Total Copies	Summary card	475.00	Total number of book copies in the catalogue
Total Book Transactions	Summary card	8	Number of transaction records
Year	Year-wise statistics	Second 1, Third 2, First 0, Fourth 2	Number of students in each year of study
Status by Fine Amount	Bar chart	Fine Amount for the statuses Returned and Issued	Compares fines on returned and issued transactions
Distribution by Status	Pie chart	Share of Issued and Returned transactions	Shows the proportion of each status
Quick Links	Link panel	Add New Book Transaction, Add New Fine Payment, Add New Book Details, Add New Student Details	Opens the four data-entry forms directly
All Book Transactions	Embedded report	8 of 8 records	Shows the transaction records on the dashboard

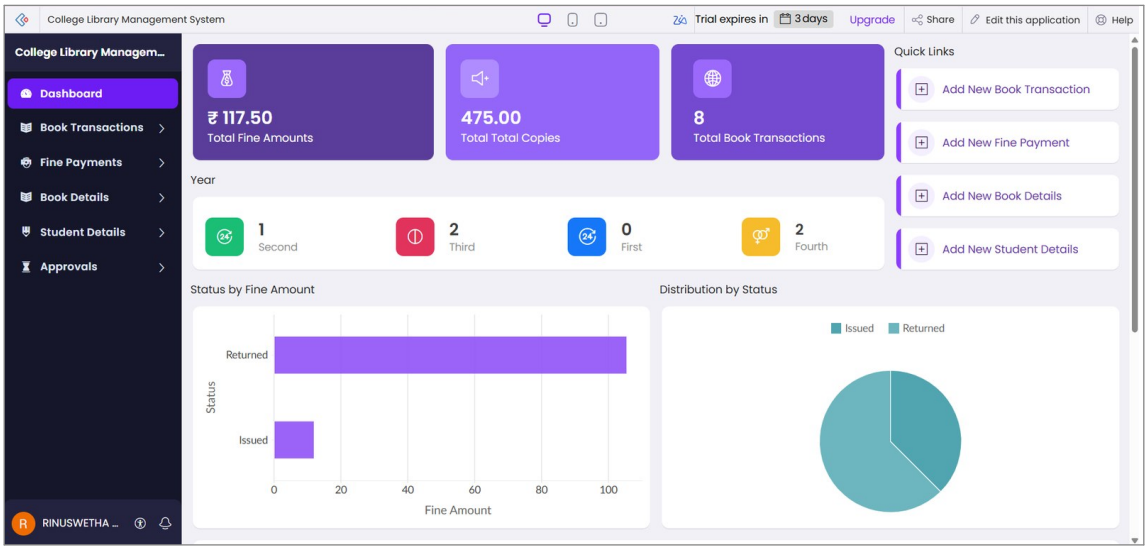


Figure 5.17: Dashboard – Summary Cards, Year-wise Statistics, Charts and Quick Links

The three summary cards agree with the reports: the fines of ₹5.50, ₹12.00 and ₹100.00 in Figure 5.11 add up to ₹117.50, the copy counts of the seven books in Figure 5.13 add up to 475, and the report holds eight transactions. The year-wise counts agree with the five students in Figure 5.15. In the Status by Fine Amount chart the Returned bar is close to 105 and the Issued bar close to 12, which matches the fines of the returned (₹100.00 + ₹5.50) and issued (₹12.00) records.

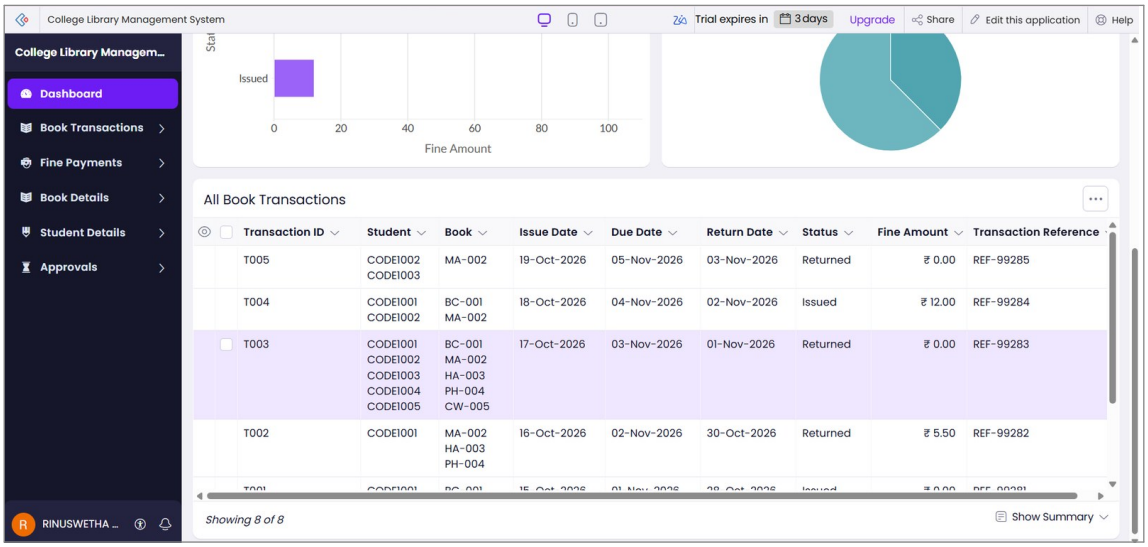


Figure 5.18: Dashboard – Status Charts and the Embedded All Book Transactions Report

5.5 Deluge Scripting

Deluge scripts were written for the ISBN lookup, the stock and return processing, the fine calculation, the scheduled fine update, the transaction e-mail and the calendar event. They are configured as actions inside form workflows and a scheduled workflow. Table 5.3 lists them and the code listings below show the logic of the main scripts; the complete code is given in Appendix C.

Table 5.3: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Book Details → On User Input of ISBN (workflow Fetch_Book_Details_API)	Fetch title, publisher and publication year from Open Library for the entered ISBN	invokeurl (GET), JSON parsing, assignment to book_name, publisher and publication_year
DS-02	Book Transaction → On Add → On Success (workflow Auto-Update Available Copies)	Reduce available copies by one for every selected book when the status is Issued	if, for each, record fetch, assignment to available_copies
DS-03	Book Transaction → return processing workflow (Status Returned and Return Processed not ticked)	Increase available copies by one for every selected book and mark the return as processed	for each, ifnull, record fetch, update of return_processed
DS-04	Function Calculate_Fine(due_date, return_date), used by Process Book Return	Calculate the overdue fine at ₹50 per overdue day when a book is returned	Custom function, date difference, arithmetic
DS-05	Book Transaction → On Schedule (workflow Bulk_Overdue_Fine_Update)	Set the fine of every overdue Issued transaction	if, daysBetween, zoho.currentdate, assignment to fine_amount
DS-06	Book Transaction → Created or Edited → Successful form submission (workflow Notify_Book_Transaction)	E-mail a notification for every saved transaction	Send mail action
DS-07	Book Transaction → Created → Successful form submission (workflow Create_Calendar_Due_Date_Event)	Create a Zoho Calendar event on the due date of the transaction	Map, put, toString, zoho.calendar.createEvent, info

DS-02 runs after a transaction is added. Because the Book field is a multi-select lookup, the script loops through every selected book ID, fetches the Book Details record and reduces Available Copies by one only if the current count is greater than zero (Code 5.1). A test transaction with the status Issued reduced the available stock by one.

Code 5.1: Stock deduction script (DS-02)

```
// DS-02 | Book Transaction | On Add -> On Success
if(input.status == "Issued" && input.book != null)
{
    for each bookID in input.book
    {
        bookDetails = book_details[ID == bookID];
        if(bookDetails.count() > 0)
        {
            if(bookDetails.available_copies > 0)
            {
                bookDetails.available_copies = bookDetails.available_copies - 1;
            }
        }
    }
}
```

DS-03 restores the stock on return. The decision box Return Processed is unticked by default; the script adds one copy for every selected book and then sets return_processed to true (Code 5.2). The available stock of a book changed from 10 to 11 after the return, and saving the same returned transaction again did not increase the stock a second time because return_processed was already true.

Code 5.2: Return processing script (DS-03)

```
// DS-03 | Book Transaction | Status = Returned and Return Processed = false
for each bookID in input.book
{
    bookDetails = book_details[ID == bookID];
    if(bookDetails != null)
    {
        bookDetails.available_copies = ifnull(bookDetails.available_copies,0) + 1;
    }
}
transactionRecord = book_transaction[ID == input.ID];
transactionRecord.return_processed = true;
```

DS-04 holds the fine rule of ₹50 per overdue day. DS-05 applies the same rule to transactions that are still issued: for every record with the status Issued and a due date, it calculates the overdue days from the due date to the current date and stores the total fine as overdue days × 50 (Code 5.3). The script sets the total fine each time it runs and does not add ₹50 again on every run.

Code 5.3: Scheduled overdue fine script (DS-05)

```
// DS-05 | Bulk_Overdue_Fine_Update | Book Transaction | On Schedule
if(input.status == "Issued" && input.due_date != null)
{
    overdueDays = daysBetween(input.due_date, zoho.currentdate);
    if(overdueDays > 0)
    {
        input.fine_amount = overdueDays * 50;
    }
}
```

DS-07 creates a calendar event for the due date of every new transaction that has a due date (Code 5.4 and Figure 5.19). The date is converted to the text format yyyyMMdd, the start and end times are set to 09:00 and 09:30 in the Asia/Kolkata time zone, and the event titled “Library Book Return Due” is created through the connection zoho_calendar_connection (Section 5.7). The response is printed with info.

Code 5.4: Calendar event script (DS-07)

```
// DS-07 | Create_Calendar_Due_Date_Event | Book Transaction | Successful form submission
if(input.due_date != null)
{
    dateText = input.due_date.toString("yyyyMMdd");

    dateTimeMap = Map();
    dateTimeMap.put("timezone","Asia/Kolkata");
    dateTimeMap.put("start",dateText + "T090000+0530");
    dateTimeMap.put("end",dateText + "T093000+0530");

    eventDetails = Map();
    eventDetails.put("estatus","added");
    eventDetails.put("title","Library Book Return Due");
    eventDetails.put("dateandtime",dateTimeMap);
    eventDetails.put("isallday","false");

    eventMap = Map();
    eventMap.put("eventdata",eventDetails);

    response = zoho.calendar.createEvent(
        "b9b926deb62b4cf1a9abb05714b62c67",
        eventMap,
        "zoho_calendar_connection"
    );

    info response;
}
```

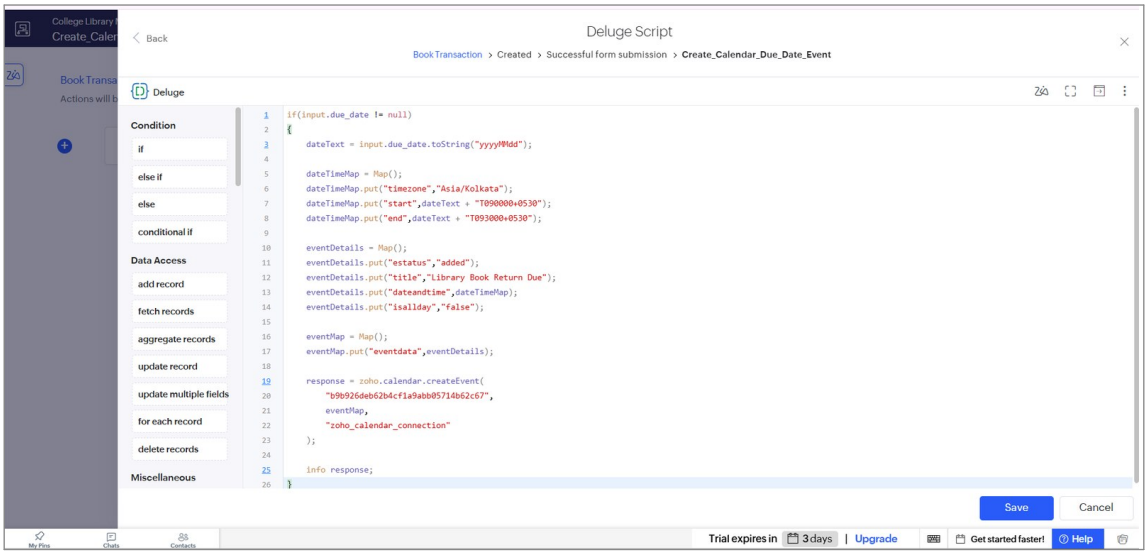


Figure 5.19: Deluge Editor with the Calendar Event Script (DS-07)

5.6 Workflows and Automation

Form workflows, a scheduled workflow and a Blueprint were used. A form workflow is created by choosing the form, the record event (Created, Edited or Created or Edited), the form event and the workflow name, and the actions are then added inside it. Table 5.4 lists the workflows implemented.

Table 5.4: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Fetch_Book_Details_API	Form workflow	Book Details → On User Input of ISBN	ISBN entered	Calls Open Library and fills book_name, publisher and publication_year	DS-01
Auto-Update Available Copies	Form workflow	Book Transaction → On Add → On Success	status is Issued and book is not null	Reduces available_copies of each selected book by one	DS-02
Return processing	Form workflow	Book Transaction → Status Returned	Return Processed not ticked	Increases available_copies by one per book and sets return_processed	DS-03
Block Issuance If No Copies	Form workflow	Book Transaction	No available copies	Stops the issue of a book that has no available copy	—
Process Book Return	Workflow using a function	Book Transaction	Book returned	Calls Calculate_Fine (₹50 per overdue day) and restores stock once	DS-03, DS-04
Bulk_Overdue_Fine_Update	Schedule	On Schedule, start 02-Nov-2026 at 10:00:00, batch size 1000, enabled	status Issued and due_date not null	Sets fine_amount to overdue days × 50	DS-05
Notify_Book_Transaction	Form workflow	Book Transaction → Created or Edited → Successful form submission	Execute without condition	Send mail (Figure 5.20)	DS-06
Create_Calendar_Due_Date_Event	Form workflow	Book Transaction → Created → Successful form submission	due_date is not null	Creates a Zoho Calendar event	DS-07
Daily overdue reminder	Schedule	Daily	Overdue transactions	Sends the overdue reminder	—
Book Issue Process	Blueprint	Book Transaction lifecycle	Transition rules	Moves a transaction through the stages of Table 5.5	—

Workflow cleanup. While the no-copies rule was being configured, several workflows named Check_Book_Availability were found; most had no actions and one had a single action, and they produced errors. The problematic duplicates were disabled and the workflow Block Issuance If No Copies was kept enabled as the rule that stops the issue of a book with no available copy.

Notification. The workflow Notify_Book_Transaction runs on every successful submission of the Book Transaction form, whether the record is created or edited, and executes without a condition. Its action sends a mail to the address shown in Figure 5.20. The notification was received and the workflow is working.

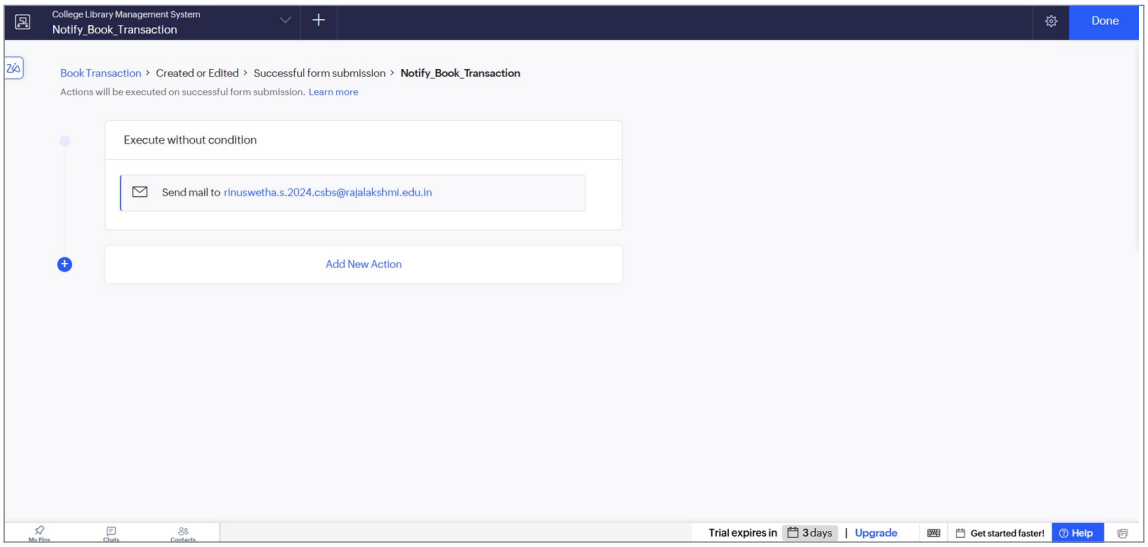


Figure 5.20: Workflow "Notify_Book_Transaction" on the Book Transaction Form

Scheduled fine update and reminder. Bulk_Overdue_Fine_Update is enabled with the start date and time 02-Nov-2026 at 10:00:00 and a batch size of 1000. The scheduled execution was run and verified. A daily overdue reminder was configured as a second scheduled workflow that finds the overdue transactions and sends the reminder; it was tested and is working.

Blueprint. The Blueprint Book Issue Process represents the life cycle of a library transaction. It was saved, enabled and verified. Table 5.5 lists its stages. The transitions between the stages are Submit for Approval, Approve Transaction, Cancel Request, Reject Transaction, Confirm Issue, Void Issue, Initiate Return, Mark Overdue, Report Book Lost, Cancel Return, Process Return, Apply Fine, Close Transaction, Process Payment, Return Book, Assess Replacement Cost and Order Replacement.

Table 5.5: Stages of the Blueprint Book Issue Process

Stage	Role in the Life Cycle
Transaction Initiated	A new borrowing transaction enters the process
Pending Approval	The transaction waits for approval
Book Issued	The issue is confirmed
Book In Use	The borrowed book is with the student
Fine Accrued	The overdue fine is applied and recorded
Fine Paid	The fine payment is completed
Return Initiated	The return process is started
Return Processed	The return operation is completed
Transaction Closed	The life cycle of the transaction is completed
Transaction Cancelled	The transaction is cancelled
Transaction Rejected	The request is rejected
Lost Book Reported	The book is marked as lost
Replacement Ordered	The replacement process is started

5.7 Integrations

Open Library. The workflow Fetch_Book_Details_API calls the public Open Library endpoint https://openlibrary.org/isbn/{ISBN}.json with the Deluge task invokeurl and the method GET. It parses the JSON response and fills Book Name, Publisher and Publication Year; Author is entered manually. For the ISBN 9783161484100 the form was filled with Logik der Forschung, Mohr Siebeck and 2005. This is an external REST API call; it is not a webhook and not a Creator REST API endpoint.

Zoho Calendar. The Creator workspace was opened and a Zoho Calendar connection was added from the list of built-in connectors (Figure 5.21). The connection is named “Zoho Calendar Connection”, the authorization access is Administrators only, the connection mode is Quick connect (Zoho credential) and the credentials may be used for all actions. Its link name is zoho_calendar_connection, its permissions are Create Event, Sync Events and Update Event, the authorization is the same for all environments, the owner is the Administrator and the authorized account is Zoho Calendar Account1. Connection access was turned on for the College Library Management System application, and the other two applications of the account were left off (Figure 5.22).

Script DS-07 (Code 5.4) uses this connection. After a transaction with a due date was saved, the event “Library Book Return Due” appeared in Zoho Calendar on Thursday 12 November 2026 at 9 am (Figure 5.23), so the integration was verified.

Table 5.6: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
ISBN autofill	Open Library – /isbn/{ISBN}.json	Deluge invokeurl, GET	Public endpoint, no connection	ISBN sent; title, publisher and publication year returned (DS-01)
Due-date calendar event	Zoho Calendar – createEvent	Creator connection zoho_calendar_connection + Deluge zoho.calendar.createEvent	Zoho connection (Quick connect, authorised by the Administrator)	Event “Library Book Return Due”, 09:00–09:30 on the due date (DS-07)
E-mail notification	Send mail task	Built-in Deluge/workflow task	Built-in, no connection	Notification for each saved transaction (DS-06)

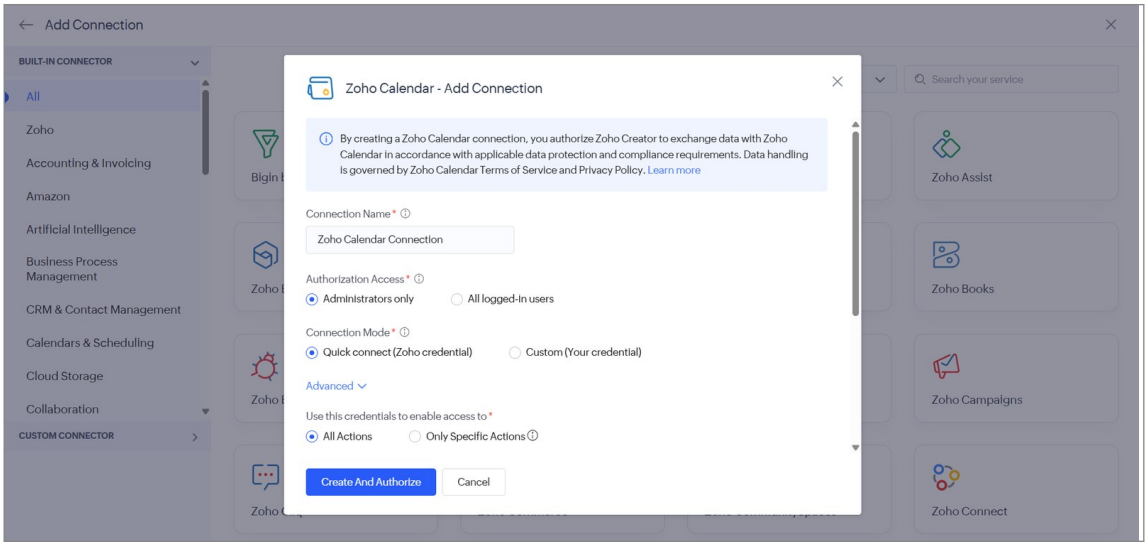


Figure 5.21: Adding the Zoho Calendar Connection

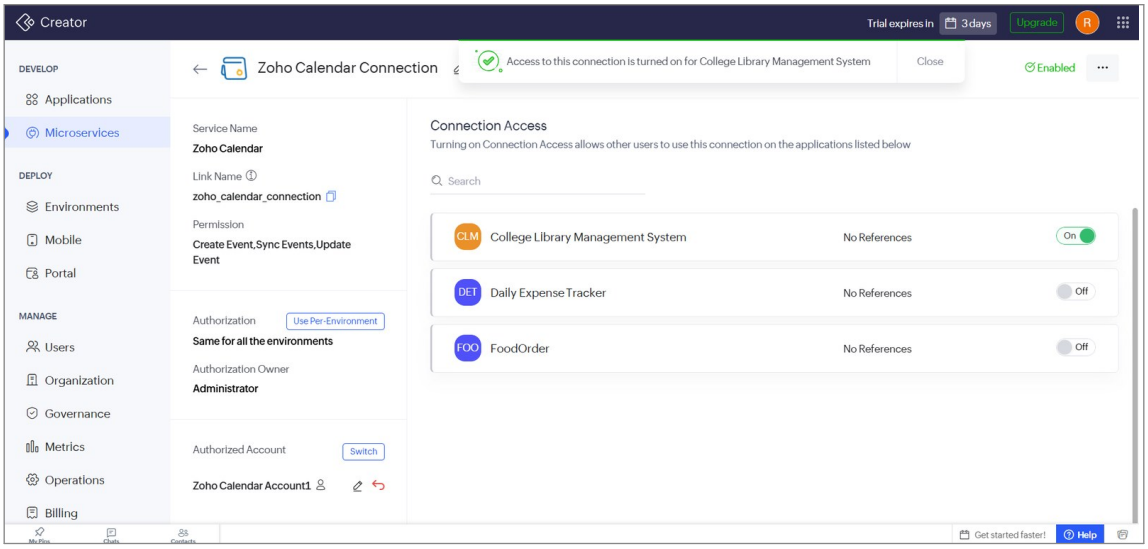


Figure 5.22: Connection Details and Access Turned On for the Application

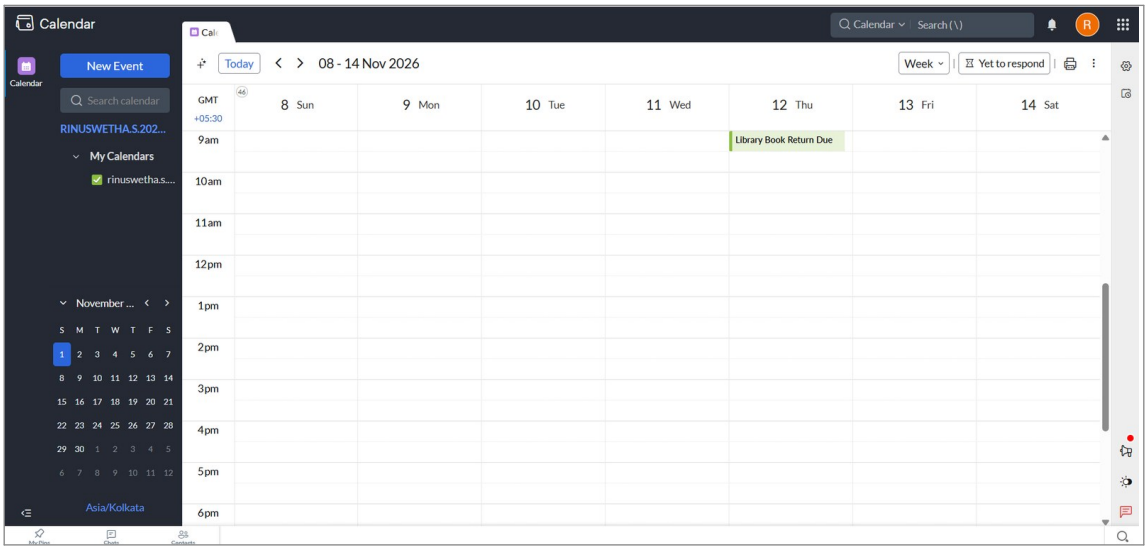


Figure 5.23: Zoho Calendar Showing the Event "Library Book Return Due"

5.8 User Management and Role-Based Access

Access is managed under Settings → Users, which has the tabs Permissions, Roles and Data Sharing. The Permissions tab lists seven permission sets (Figure 5.24; Table 5.7): the two built-in sets Read and Write, and five custom sets, Admin Access, Librarian, Cataloging Specialist, Fines Administrator and Student Registrar. Each custom set carries a short description of its purpose.

Table 5.7: Permission Sets

Permission Set	Description (as shown)	Module Settings
Admin Access	Full access to manage books, students, transactions, fine payments and dashboard	Access ticked for Book Details, Book Transaction, Fine Payment, Student Details and Dashboard; View, Edit and Delete on all reports of the four forms; More: Import & Export/Print (Figure 5.25)
Librarian	Manage the issuance and return of books to students	Access ticked for the same five modules; View and Edit on Book Details (2 reports), All Book Transactions and All Fine Payments; View only on Student Details (2 reports); Delete and More not ticked (Figure 5.26)
Cataloging Specialist	Create and maintain detailed records for new book acquisitions	Listed in Figure 5.24
Fines Administrator	Calculate and apply fines for overdue book returns	Listed in Figure 5.24
Student Registrar	Register new students and verify their department details	Listed in Figure 5.24
Read	Read permission for all components	Built-in set
Write	Write permission for all components	Built-in set

Admin Access. Figure 5.25 shows the module permissions of Admin Access. All five modules have Access ticked, the View, Edit and Delete columns cover all reports of the four forms (for example All Book Transactions and All Fine Payments), and the More column offers Import &

Export/Print. The Dashboard row has Access only.

Librarian. Figure 5.26 shows the Librarian set. The same five modules have Access ticked and the View and Edit columns list Book Details (2 reports), All Book Transactions and All Fine Payments. Student Details can be viewed (2 reports) but its Edit box is not ticked, and Delete and More are not ticked for any module. A librarian can therefore issue and return books and record fines but cannot delete records or change student records.

Roles and data sharing. The Roles tab holds the hierarchy CEO → Admin → Librarian → Student (Figure 5.27), with the Student role at the lowest level. The Data Sharing tab holds four share rules that share records from the role Admin to the role Librarian (Figure 5.28): Share_Books_With_Libr… (Book Details, Read/Write), Share_Transaction_Wit… (Book Transaction, Read/Write), Share_Fines_With_Libra… (Fine Payment, Read/Write) and Share_Students_With_L… (Student Details, Read Only). Together, the permission sets, the role hierarchy and the share rules separate the duties of the administrator, the librarian and the student according to the principle of least privilege. The role and permission configuration was tested with Admin, Librarian and Student accounts and verified by the student.

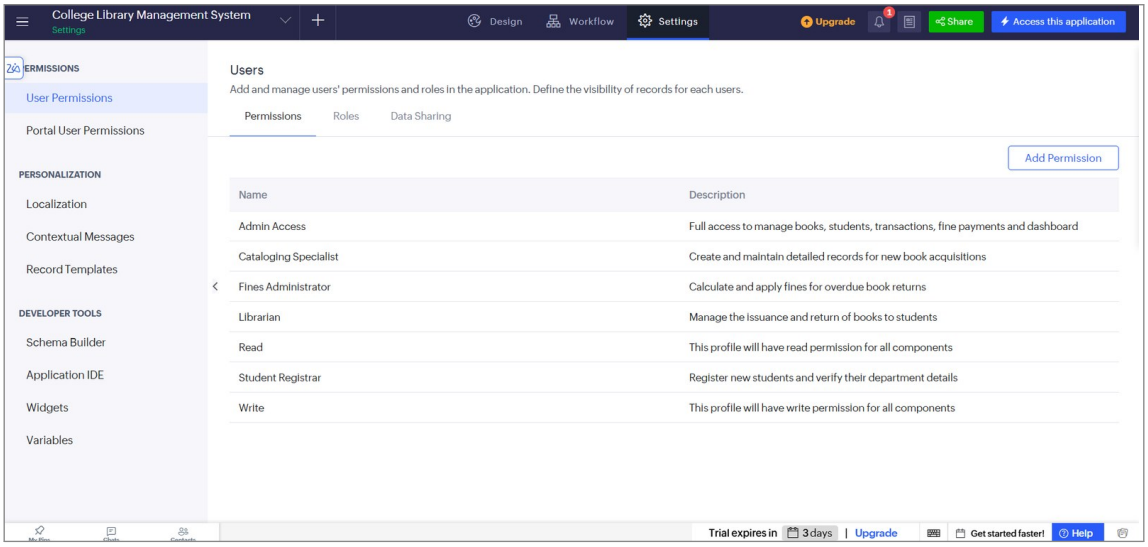


Figure 5.24: Permission Sets of the Application

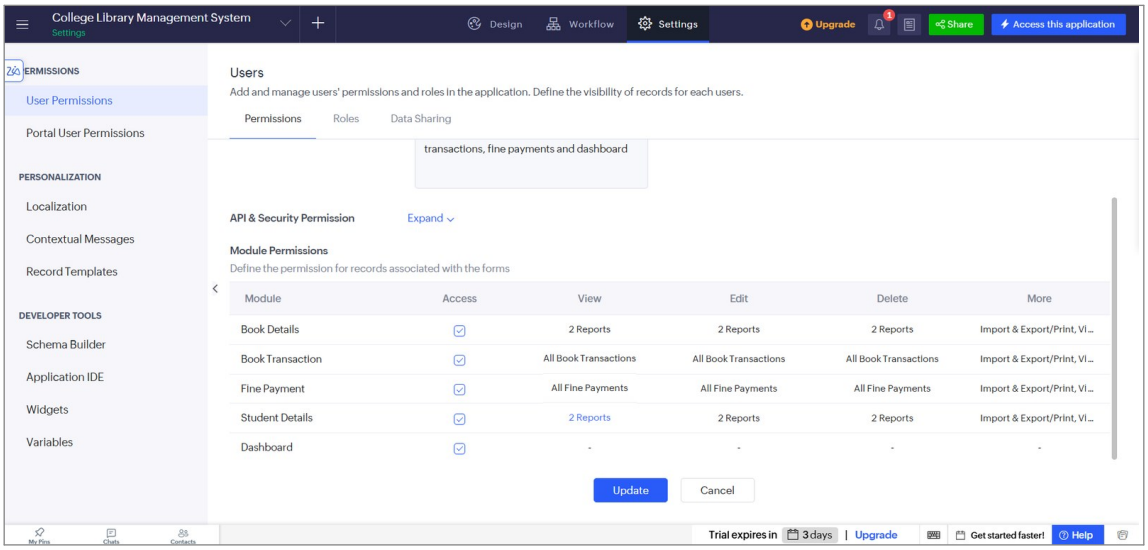


Figure 5.25: Admin Access Permission Set

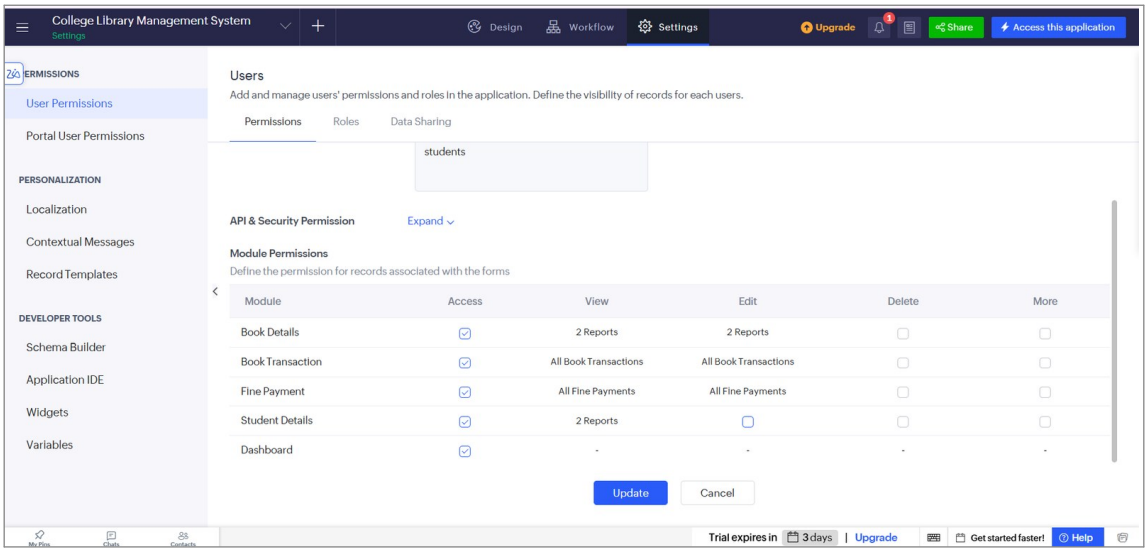


Figure 5.26: Librarian Permission Set

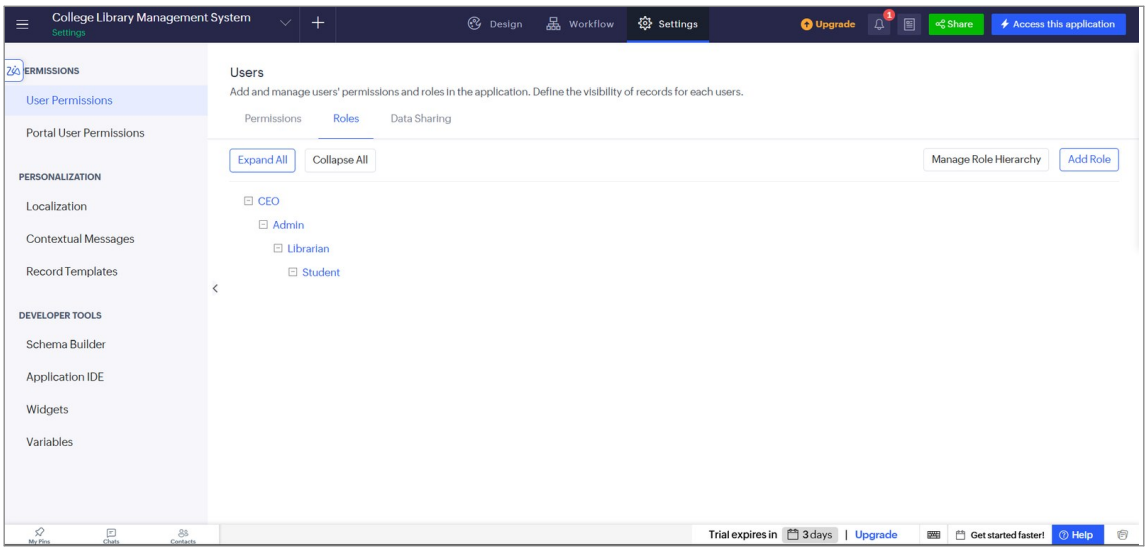


Figure 5.27: Role Hierarchy: CEO → Admin → Librarian → Student

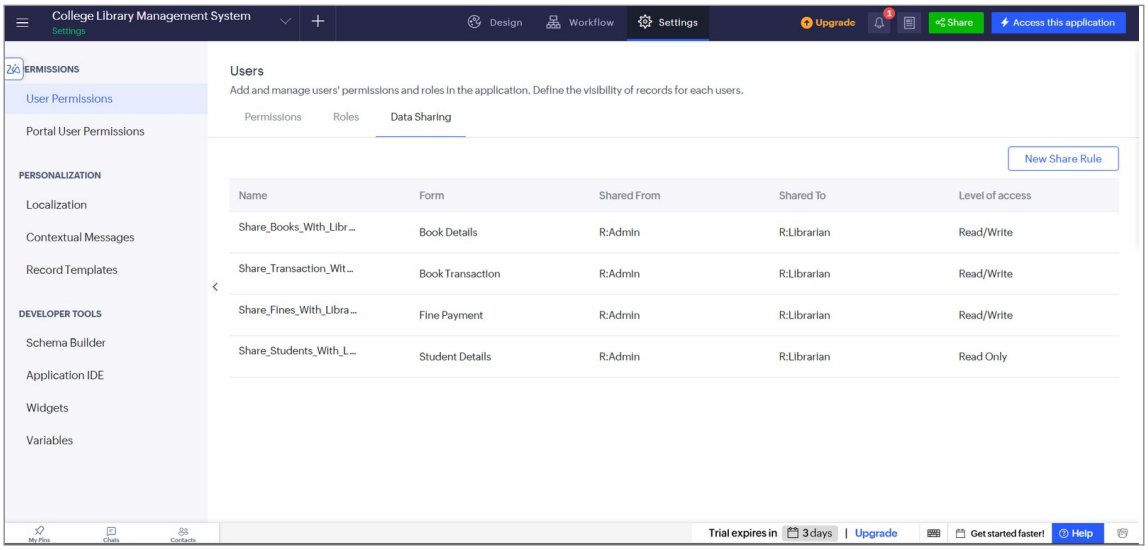


Figure 5.28: Data Sharing Rules from Admin to Librarian

5.9 AI and Analytics Features

Analytics. The analytics of the application are the reports of Section 5.3 and the dashboard of Section 5.4: the summary cards for total fine amount, total copies and total transactions, the year-wise student count, the bar chart Status by Fine Amount, the pie chart Distribution by Status, and the Kanban reports Categories and Departments, which group books by category and students by department. Zoho Analytics was not integrated.

AI features. No Zia or other AI feature of Zoho Creator (such as AI fields, prediction, OCR or sentiment analysis) was configured as a runtime feature of the delivered application. The intelligence of the application lies in its Deluge automation: stock updates, fine calculation, the scheduled fine update and the reminders.

5.10 Pro-Code Extension

No pro-code extension such as Zoho Catalyst, a JavaScript widget or an external service was used, and the Microservices section of the Creator workspace contains no microservice (Figure 5.29). The required behaviour – catalogue and student records, stock handling, fines, notifications and role-based access – was covered by forms, workflows, the Blueprint and Deluge scripts, which form the low-code scripting layer of the application. The Open Library call is an external REST call made with invokeurl and is not a use of the Creator REST API. A future extension could expose the Book Transaction form through the Creator REST API so that a barcode or RFID scanner at the library desk could issue and return books.

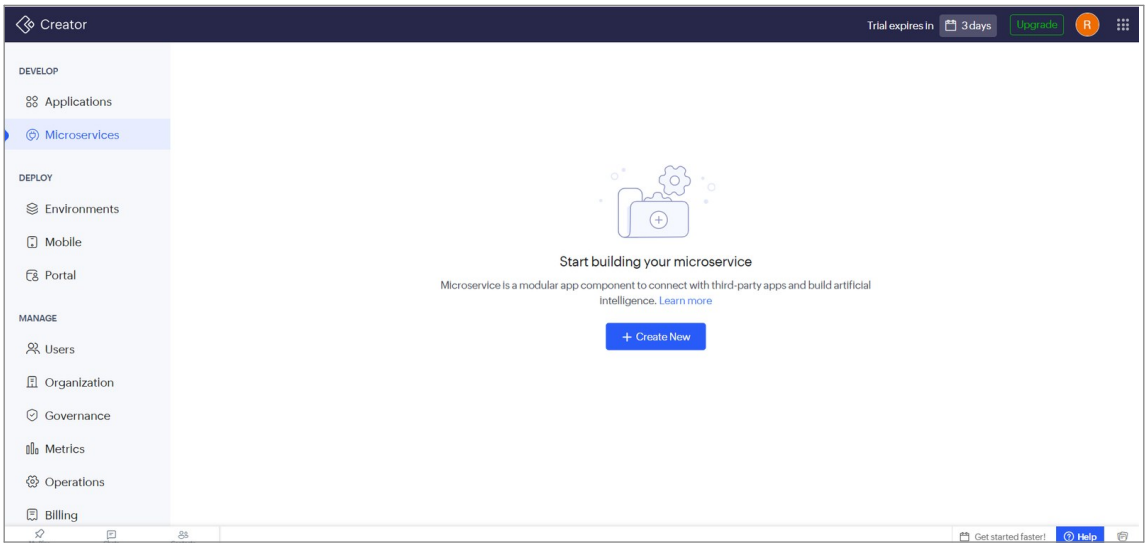


Figure 5.29: Microservices Section of the Creator Workspace (no microservice created)

5.11 Mobile and Portal Access

The Creator workspace lists Mobile and Portal under Deploy (Figure 5.29), and the application header offers desktop, tablet and phone preview toggles. A dedicated mobile configuration and a separate portal for students were not set up for this application, so the application is used through the web browser with the Admin, Librarian and Student roles of Section 5.8. A student portal for checking available books and fines from a phone is listed as a future enhancement.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done on the live application in Zoho Creator while it was being built and again after the workflows, the Blueprint and the role settings were completed. Unit testing checked each form by entering sample data (five students, seven books, nine transactions and three fine payments) and each report and dashboard component by comparing it with the stored records. Integration testing checked the workflows by adding and editing book transactions and observing the available copies, the fine amount, the e-mail and the calendar event. The scheduled fine update, the daily overdue reminder and the Book Issue Process Blueprint were run and verified by the student. Role-based testing was done by using the application with Admin, Librarian and Student role accounts (Section 5.8). User acceptance testing with other users has not been recorded (Section 6.5).

Table 6.1 refers to the functional requirements in the order in which they are listed in Chapter 3: FR-01 book catalogue, FR-02 student records, FR-03 book issue, FR-04 book return, FR-05 stock management, FR-06 overdue fine, FR-07 fine payment, FR-08 ISBN autofill, FR-09 e-mail and reminder, FR-10 reports and dashboard, and FR-11 roles and access.

6.2 Test Cases

The test cases are listed in the table that follows. Each row carries the test case number (TC ID), the functional requirement it verifies (FR Ref.), a short description, the input or steps used, the expected result, the actual result observed in the application, and the final status. Nineteen test cases (TC-01 to TC-19) were prepared for the Smart Library Management System, and every functional requirement from FR-01 to FR-11 is covered by at least one of them. The cases are grouped below by the area of the application they exercise.

Table 6.1a: Test Case Coverage by Application Area

Application area	Test cases	Total	Passed
Catalogue and student records	TC-01, TC-02, TC-04	3	3
ISBN autofill (Open Library)	TC-03	1	1
Issue, return and stock	TC-05 to TC-09	5	4
Fines and fine payments	TC-10 to TC-12	3	3
E-mail, reminder and calendar	TC-13 to TC-15	3	3
Reports and dashboard	TC-16, TC-17	2	2
Blueprint and role-based access	TC-18, TC-19	2	2
Total	TC-01 to TC-19	19	18

Table 5.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add a book to the catalogue	Book Name, Author, ISBN, Category, Publisher, copies, year	Record saved	Seven books are listed in the Book Details Report (Fig. 5.13)	Pass
TC-02	FR-01	Mandatory fields left empty on Book Details	Submit the form with ISBN, Category and Publication Year empty	Form shows an error and does not save	“Invalid entries found – Rectify and submit again”; the three fields are marked as mandatory (Fig. 6.4)	Pass
TC-03	FR-08	ISBN autofill from Open Library	Enter ISBN 9783161484100	Book name, publisher and year are filled	Logik der Forschung; Mohr Siebeck; 2005	Pass
TC-04	FR-02	Add a student record	Student Name, Register Number, e-mail, department, year, phone, code	Record saved	Five students are listed in All Student Details (Fig. 5.15)	Pass
TC-05	FR-03	Issue a book to a student	New transaction: student, book, issue date, due date, status Issued	Transaction saved and listed	Saved; a ninth record (CODE1003, CW-005, due 12-Nov-2026) appears in All Book Transactions (Fig. 6.1)	Pass
TC-06	FR-05	Stock deduction on issue	Add a transaction with status Issued and a book selected	Available Copies reduced by one	Available stock decreased by one	Pass
TC-07	FR-05	Issue blocked when no copy is available	Try to issue a book whose Available Copies is zero	Issue is blocked by Block Issuance If No Copies	The workflow is enabled; the result of a zero-stock attempt was not recorded	Not recorded
TC-08	FR-04	Stock restoration on return	Mark the transaction Returned with Return Processed unticked	Available Copies increased by one	Stock changed from 10 to 11	Pass
TC-09	FR-04	Duplicate return protection	Edit and save the same returned transaction again	Stock not increased again	Stock stayed at 11 because return_processed was already true	Pass
TC-10	FR-06	Fine for an overdue return	Due date 29-Oct-2026, returned on 31-Oct-2026	2 days × ₹50 = ₹100	₹100.00 shown in Fine Amount (Fig. 6.1)	Pass
TC-11	FR-06	Scheduled fine update	Run Bulk_Overdue_Fine_Update for overdue Issued transactions	Fine Amount set to overdue days × 50	Scheduled execution run and verified by the student	Pass
TC-12	FR-07	Record fine payments	Add payments with status Paid or Pending and method Cash or Online	Records saved and listed	Three payments are listed in All Fine Payments (Fig. 5.12)	Pass
TC-13	FR-09	Transaction e-mail	Save a book transaction	E-mail alert received	E-mail “New Book Transaction Alert” received from notifications@trial.zohocreatormail.com (Fig. 6.2)	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-14	FR-09	Daily overdue reminder	Let the daily schedule run for overdue transactions	Reminder sent	Reminder received; workflow working	Pass
TC-15	FR-09	Due-date calendar event	Save a transaction with a due date	Event “Library Book Return Due” in Zoho Calendar	Event shown on 12 Nov 2026 at 9 am (Fig. 5.23), the due date of the ninth transaction	Pass
TC-16	FR-10	Grouped reports	Open Categories and Departments	Records grouped by category and department	Books and students shown in their columns (Fig. 5.14, 5.16)	Pass
TC-17	FR-10	Dashboard values match the records	Open the Dashboard	Cards, charts and counts agree with the reports	9 transactions, ₹117.50 fines, 475 copies, Returned = 5 in the pie chart (Fig. 6.3)	Pass
TC-18	FR-03, FR-04	Blueprint life cycle	Run a transaction through the Book Issue Process transitions	Transitions move the stages in order	Blueprint completed and verified by the student	Pass
TC-19	FR-11	Role-based access	Use Admin, Librarian and Student accounts	Each role sees only the permitted modules and actions	Access matched the permission sets (Section 5.8); verified by the student	Pass

Eighteen of the nineteen test cases passed. TC-07 is recorded as “Not recorded” because the workflow is enabled but the outcome of a zero-stock attempt was not captured.

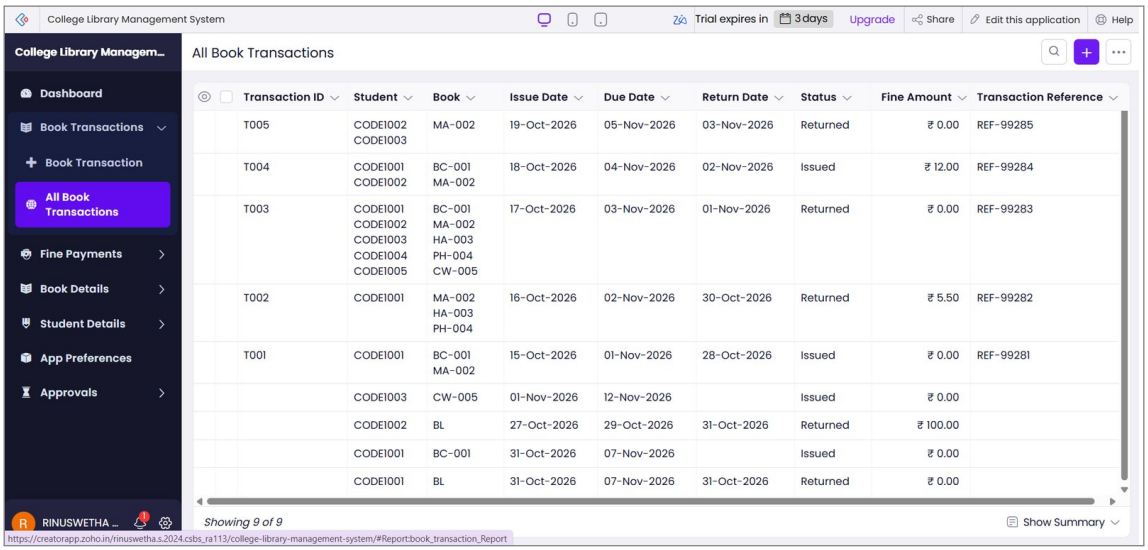


Figure 5.1: All Book Transactions Report with Nine Records (TC-05, TC-10)

In Figure 5.1 the ninth transaction (CODE1003, CW-005, due 12-Nov-2026) is the record saved in TC-05, and the Fine Amount of ₹100.00 on the returned transaction is the result of TC-10.

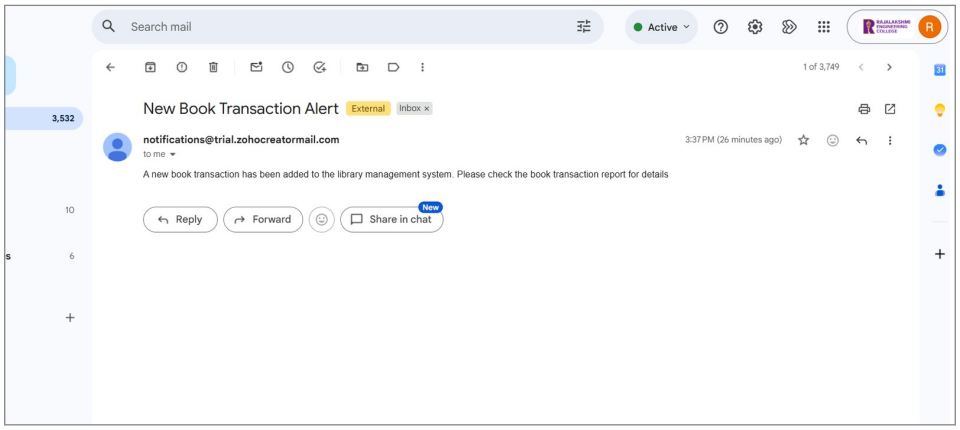


Figure 5.2: E-mail “New Book Transaction Alert” Received in the Inbox (TC-13)

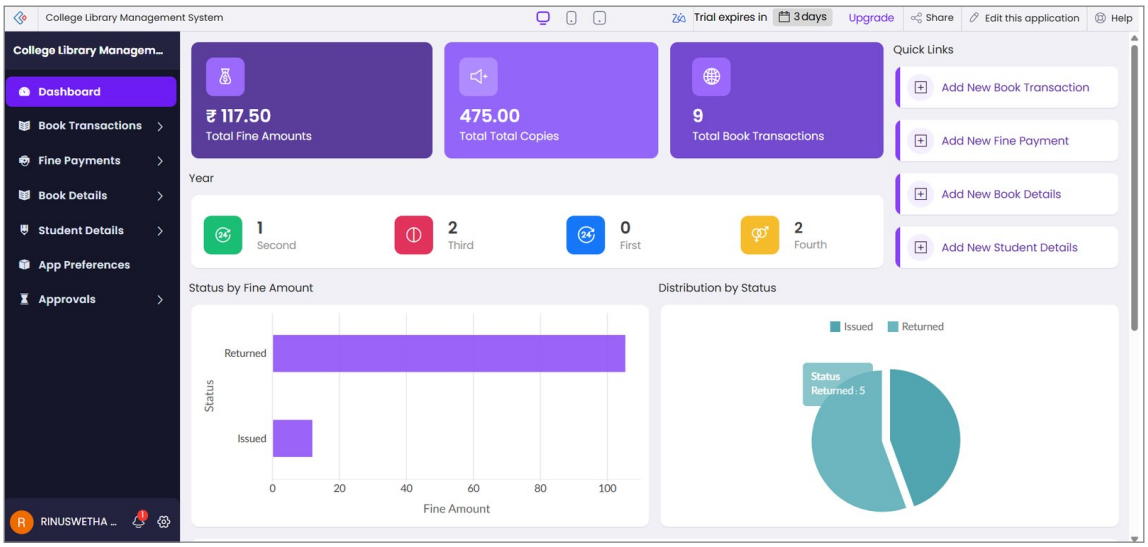


Figure 5.3: Dashboard after the Test Transactions (TC-17)

6.3 Debugging Deluge Scripts and Workflows

Debugging relied on five techniques: the error message shown by a form on submission, the available copies and fine amounts of the stored records before and after a test, the response written to the log by the info statement at the end of the calendar script DS-07, the e-mail and the calendar event that actually arrived, and the Enabled status of each workflow in the workflow list, which confirms that the script was saved without a syntax error. Figure 6.4 shows a form error message: when Book Details is submitted with the mandatory fields ISBN, Category and Publication Year empty, the form lists the invalid entries and does not save the record.

Duplicate availability workflows. While the no-copies rule was being configured, several workflows named Check_Book_Availability were present. Most of them had no actions and one had a single action, and they produced errors. The problematic duplicates were disabled and the workflow Block Issuance If No Copies was kept enabled (BUG-01).

Repeated stock restoration. Editing and saving a returned transaction a second time could add the returned copies again. The decision box Return Processed was added to the Book Transaction form with the default unticked; the return script now sets it to true after restoring the stock, and the stock stayed at 11 on the second save (BUG-02).

Wrong book selected in a test. During one issue test a book other than the intended one was selected in the Book lookup, so the expected book showed no change. The selection was corrected and the stock deduction was confirmed on the next test (BUG-03).

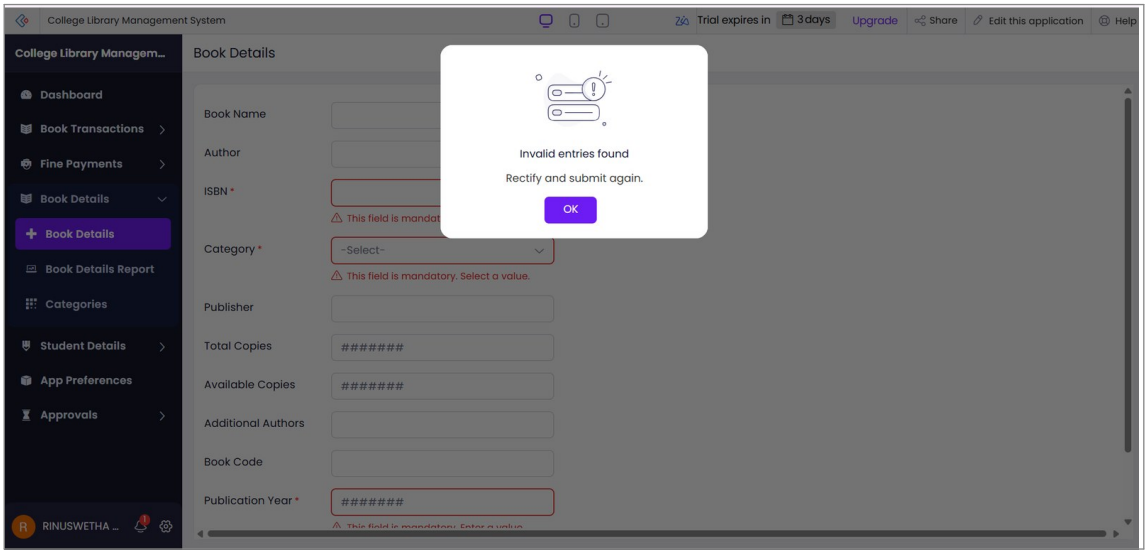


Figure 5.4: Form Validation Message on the Book Details Form (TC-02)

6.4 Defect Log

The defect log records every problem found while the forms, workflows, Blueprint and reports were being built and tested. For each defect it gives the bug number, a description of the symptom, the root cause, the fix applied and the present status. Six defects, BUG-01 to BUG-06, are recorded in the table that follows.

Table 6.2a: Defect Status Summary

Status	Count	Defects	Nature of the defect
Closed	3	BUG-01, BUG-02, BUG-03	Workflow configuration, stock-processing logic and a wrong book selection in a test
Open	2	BUG-04, BUG-05	Missing fine reset and missing Available Copies validation; fixes are proposed
Accepted	1	BUG-06	Dashboard card label kept as built; renaming is proposed

The three closed defects were corrected during development and re-tested. The two open defects are data checks outside the current scope and do not affect the verified test results.

Table 5.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Errors while saving the availability rule; several Check_Book_Availability workflows existed	Duplicate workflows, most with zero actions	Problematic duplicates disabled; Block Issuance If No Copies kept enabled	Closed
BUG-02	Saving a returned transaction again could increase stock again	The return script had no record that the return was already processed	Decision box return_processed added; stock stayed at 11 on re-save	Closed
BUG-03	Stock did not change in one issue test	A different book was selected in the Book lookup	Correct book selected; deduction confirmed in the next test	Closed
BUG-04	Fine Amount is not reset when a transaction is no longer overdue	The scheduled script sets fine_amount only when overdue days are greater than zero	Accepted for the current scope; a reset is proposed	Open
BUG-05	Available Copies can be larger than Total Copies (Logik der Forschung shows 12 of 10, Fig. 5.13)	The return script adds one copy without comparing with Total Copies, and the form has no check	Left as test data; a validation is proposed	Open
BUG-06	Dashboard card is labelled “Total Total Copies”	Label of the card as built	Kept as built; renaming is proposed	Accepted

6.5 User Acceptance Testing

No feedback from other users has been recorded for this application, so no participant names or comments are given. The student tested the application with the three roles of the library, and Table 6.3 records this check.

Table 5.3: User Acceptance Testing Record

Role Tested	Tested By	Activities Tried	Result
Admin	Student (developer)	Managing books, students, transactions, fine payments and the dashboard	Access matched the Admin Access permission set; verified
Librarian	Student (developer)	Issuing and returning books, recording fines, viewing student records	Access matched the Librarian permission set; verified
Student	Student (developer)	Using the permitted student-facing functions	Access limited to the permitted functions; verified

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator provides a multi-stage deployment space (Deploy → Environments) with Stage and Production environments, while development takes place in the builder. For the library application, the Environments page shows “No changes available”, which means that the version in the builder is identical to the version already published. Version 1.0 is listed in Stage (dated Nov 01, 2026) and version 1.0 is listed in Production (also dated Nov 01, 2026 in the later capture). The same page lists the application under the display name “College Library Management System”.

The publish menu offers “Publish to Stage” and “Publish to Production”, and a counter beside the Production option reads 0/30. Another application in the same account (FoodOrder) shows “Changes available” and “Not yet published” in both Stage and Production; it belongs to a different project and was not touched. The Environments page also provides a Version History button, which is the basis of the rollback plan: the previous version can be republished if a later change breaks the application. No rollback is recorded for this project.

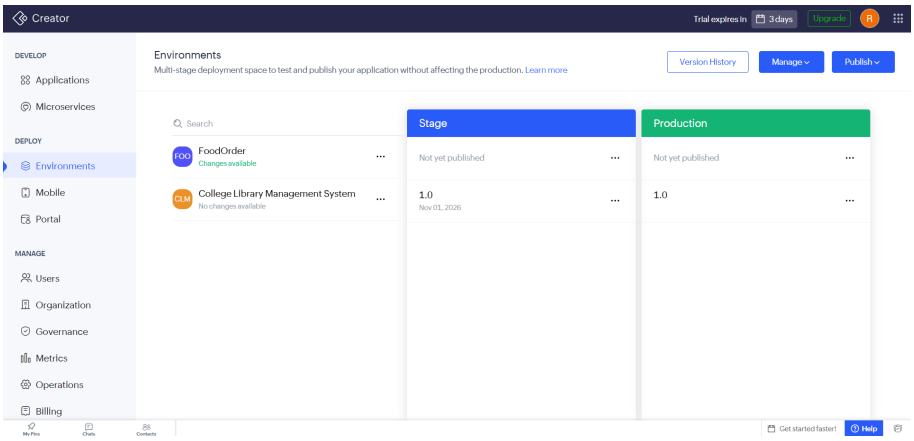


Figure 7.1: Environments Page Listing the Application in Stage and Production

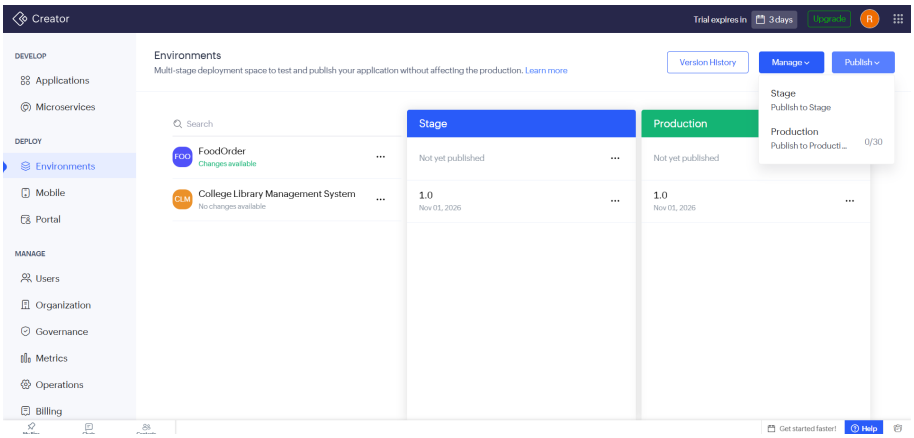


Figure 7.2: Publish Menu: Publish to Stage / Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Application published to Stage. (Version 1.0, Nov 01, 2026, Figure 7.1.)	✓
2	Application published to Production. (Version 1.0, Nov 01, 2026, Figure 7.2.)	✓
3	Environments page shows no pending changes. (“No changes available” for the library application.)	✓
4	Roles and permission settings (Admin, Librarian, Student) configured and verified by the student.	✓
5	Scheduled and notification workflows enabled. (Bulk_Overdue_Fine_Update enabled, batch size 1000, start 02-Nov-2026 10:00:00; transaction email and daily overdue reminder working.)	✓
6	Book Issue Process Blueprint saved, enabled and verified.	✓
7	Duplicate Check_Book_Availability workflows disabled; Block Issuance If No Copies retained.	✓
8	Published application URL recorded. ([INSERT SCREENSHOT / COPY EXACT LABEL])	Partial
9	Release notes recorded for version 1.0. (Not recorded.)	✗

7.2 Performance Considerations

No timing or load measurements were taken, so no performance improvement is claimed. The design choices that help keep the application light are: the ISBN lookup is called only when the ISBN field is edited; the stock scripts iterate only over the books selected in the transaction; the scheduled fine update is processed in batches of 1000 records; the scheduled script considers only transactions with status Issued and a non-null due_date; and the fine is set as the total overdueDays × ₹50, so repeated schedule runs give the same result instead of accumulating. Table 7.2 records these

choices honestly.

Table 7.2: Performance Considerations

Design Choice	Component	Effect	Measured Result
Lookup on ISBN input only	Fetch_Book_Details_API	External call made only when the ISBN is entered	Not measured
Loop over selected books only	Auto-Update Available Copies	Updates only the books in the transaction	Not measured
Batch size 1000	Bulk_Overdue_Fine_Update	Overdue fines processed in bounded batches	Not measured
Total fine, not cumulative	Bulk_Overdue_Fine_Update	Repeated runs give the same fine value	Functional; not timed
return_processed guard	Return processing workflow	Prevents repeated stock updates on re-save	Verified (10 → 11, no second increment)

7.3 Security Features

Authentication is by the account sign-in of the platform, and access to application data is controlled through the Admin, Librarian and Student roles and their permission settings. The student configured and personally verified these roles. The separation of Admin, Librarian and Student responsibilities follows the principle of least privilege. Table 7.3 lists the controls implemented and Table 7.4 summarises the role model.

Table 7.3: Security and Integrity Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based permissions	Admin, Librarian and Student roles with permission settings (exact profile labels: [INSERT SCREENSHOT / COPY EXACT LABEL])	Unauthorised viewing or changes to records
Block Issuance If No Copies	Book Transaction (issue)	Issuing a book with no available copies
return_processed Decision Box	Book Transaction (return processing)	Duplicate stock restoration on re-save
Book Issue Process Blueprint	Book Transaction lifecycle (13 stages)	Uncontrolled movement between issue, return, fine and closure states
Public API without credentials	Open Library ISBN GET through invokeurl	Storing keys or passwords in scripts; the scripts contain none

Security Control	Where Applied	Threat Mitigated
Disabled duplicate workflows	Check_Book_Availability duplicates	Conflicting or failing validation logic

Table 7.4: Role Model of the Application

Role	Intended Responsibility	Access Model
Admin	Application and library administrator	Manages application configuration and library records
Librarian	Performs library operations	Manages catalogue, issue and return transactions, fines and related reports
Student	Library borrower	Uses permitted student-facing functions; views permitted book and borrowing information; no administration

The exact per-form and per-report permission toggles are to be reproduced from the completed permission screens: [INSERT SCREENSHOT / COPY EXACT LABEL]. Field-level encryption, IP or domain restrictions, two-factor authentication and audit-log settings were not evidenced in the captured screens and are therefore not claimed. Passwords and credentials are not printed in this report.

7.4 Governance and Administration

Administration is carried out from the Deploy → Environments page and from the Manage section of the left menu, which lists Users, Organization, Governance, Metrics, Operations and Billing (Figure 7.1). Naming follows a simple convention: one form per entity (Book Details, Student Details, Book Transaction, Fine Payment) and workflows named after what they do (Fetch_Book_Details_API, Auto-Update Available Copies, Bulk_Overdue_Fine_Update).

For data quality, the return_processed flag keeps stock counts consistent, and the scheduled job recalculates the fine from the due date instead of adding to it. For workflow maintenance, problematic duplicate availability workflows were disabled and the intended Block Issuance If No Copies workflow was retained. The dashboard was kept as built, including the duplicated Total Fine Amounts card and the “Total Total Copies” label; these are recorded as known labelling issues, not as changes made. The page header also shows “Trial expires in 3 days” with an Upgrade button, so continued access after the trial depends on the plan of the account. No audit trail or backup configuration was evidenced.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The working application is presented through one library scenario that follows the records from the catalogue to the dashboard. In step 1 the Book Details Report lists the catalogue with the ISBN, category, publisher, total copies and available copies of each book (Figure 8.1). In step 2 the All Book Transactions report shows the issue date, due date, return date, status and fine amount of the nine transactions (Figure 8.2). In step 3 the e-mail “New Book Transaction Alert”, sent when a transaction is saved, is received in the inbox (Figure 8.3). In step 4 the dashboard summarises the fines, copies, transactions and statuses (Figure 8.4).

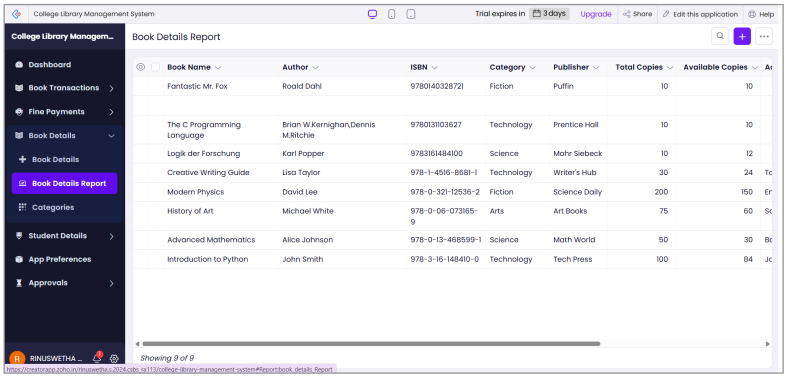


Figure 8.1: Scenario Step 1 – Book Details Report Showing the Catalogue and Copy Counts

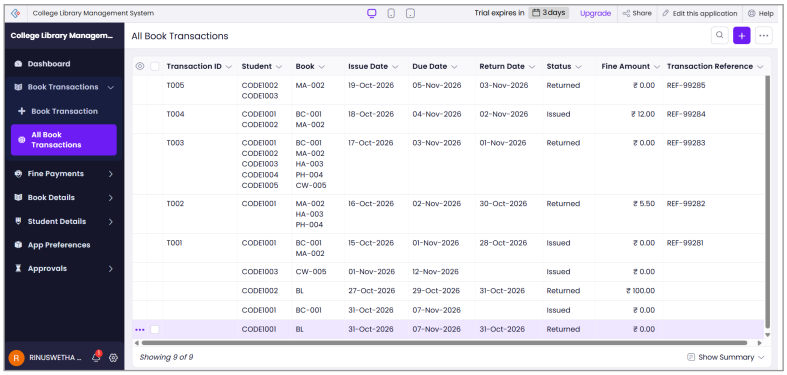


Figure 8.2: Scenario Step 2 – All Book Transactions Report with Nine Records

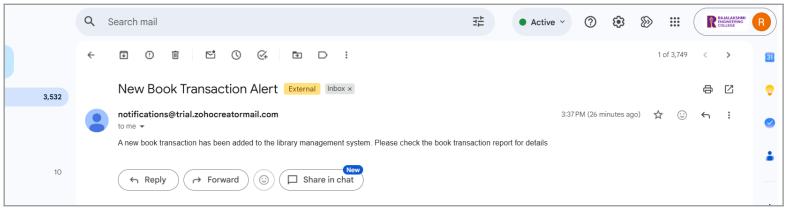


Figure 8.3: Scenario Step 3 – E-mail Alert Received in the Inbox

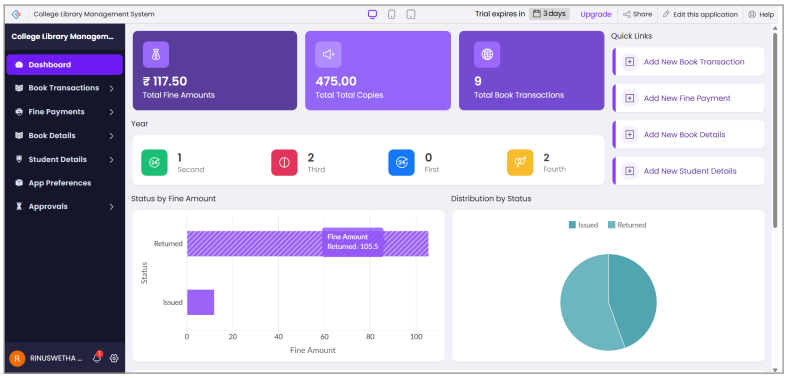


Figure 8.4: Scenario Step 4 – Dashboard with Fine Amounts, Copies, Transactions and Status Charts

The four figures agree with one another and with the test results of Chapter 6. The Fine Amount column of Figure 8.2 holds ₹100.00, ₹12.00 and ₹5.50, which add up to the ₹117.50 shown on the first dashboard card in Figure 8.4. The fines of the Returned transactions add up to 105.5, the value shown by the Returned bar in the chart Status by Fine Amount, and the Issued bar corresponds to the ₹12.00 fine on an Issued transaction. Figure 8.2 lists five Returned and four Issued transactions, and the year-wise statistics in Figure 8.4 (First 0, Second 1, Third 2, Fourth 2) add up to the five student records. Figure 8.1 also shows the book Logik der Forschung with the ISBN 9783161484100 and the publisher Mohr Siebeck, the ISBN that was used for the autofill test, with 10 total copies and 12 available copies, the case recorded as BUG-05.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
1. Catalogue of books with ISBN, publisher, year and copy counts	Sections 4.3, 5.2	Achieved. Book Details form with mandatory-field validation (TC-01, TC-02) and the Book Details Report (Fig. 8.1).
2. Student records linked to borrowing transactions	Sections 4.2, 5.2	Achieved. Student Details form (TC-04) and the Student lookup in Book Transaction; student codes are shown in Fig. 8.2.
3. Recording of book issue, due date, return and status	Sections 4.3, 5.2	Achieved. Nine transactions with issue, due and return dates and status in Fig. 8.2 (TC-05, TC-08).

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
4. Stock management: deduction on issue, restoration on return, no-copies rule	Sections 5.5, 5.6	Achieved for deduction and one-time restoration (TC-06, TC-08, TC-09). The no-copies workflow is enabled, but the result of a zero-stock attempt was not recorded (TC-07).
5. Overdue fine at ₹50 per day and fine payment records	Sections 5.5, 5.6	Achieved. ₹100.00 for two overdue days (TC-10, Fig. 8.2), scheduled fine update verified (TC-11), three payments recorded (TC-12).
6. Book details from the Open Library ISBN service	Section 5.7	Achieved. ISBN 9783161484100 filled Logik der Forschung, Mohr Siebeck and 2005 (TC-03).
7. Transaction e-mail and daily overdue reminder	Section 5.6	Achieved. E-mail received (Fig. 8.3, TC-13); reminder and calendar event verified (TC-14, TC-15).
8. Reports and dashboard	Sections 5.3, 5.4	Achieved. Reports in Fig. 8.1 and 8.2; dashboard cards, year-wise statistics and charts in Fig. 8.4 (TC-16, TC-17).
9. Roles and Blueprint lifecycle	Sections 4.6, 5.6, 5.8	Achieved. Admin, Librarian and Student access and the Book Issue Process Blueprint verified by the student (TC-18, TC-19).

8.3 Comparison with the Existing System

No timing or error-rate measurements were made for either the manual process or the application, so Table 8.2 compares them qualitatively and no percentage improvements are claimed.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Recording books and students	Separate registers or sheets; structure varies	Book Details and Student Details forms with mandatory fields and lookups	Consistent, complete records
Entering a book	Title, publisher and year typed by hand	ISBN autofill from Open Library for name, publisher and year; author typed	Less manual typing
Stock after issue and return	Copies counted and corrected by hand	Available Copies reduced on issue and restored once on return	Stock updated without manual counting
Overdue fines	Days counted and fine worked out by hand	Fine of overdue days × ₹50, updated by a scheduled batch	One fine rule applied the same way (₹100.00 for 2 days)

Parameter	Existing System	Proposed Application	Improvement
Notification and reminders	Staff must remember to inform or chase	E-mail alert on each transaction and a daily overdue reminder	Overdue records flagged without manual steps
Summary reporting	Counting by hand	Reports, dashboard cards and charts	Fines, copies and statuses on one page
Access and control	Whoever holds the register sees everything	Admin, Librarian and Student roles with permission sets	Responsibilities separated by role

Table 8.3 records the quantities that can be read from the report and dashboard screenshots (Figures 8.1 to 8.4). They describe the sample data at the time of the screenshots and are observations, not performance measurements.

Table 8.3: Observed Application State from the Screenshots

Measure	Observed Value	Source
Records in the Book Details Report	9 (Showing 9 of 9); 8 rows show book details	Figure 8.1
Book transactions in the application	9	Figure 8.2, Figure 8.4
Transactions with status Returned	5	Figure 8.2
Transactions with status Issued	4	Figure 8.2
Total fine amount	₹117.50	Figure 8.4 (card)
Fine amount of Returned transactions	105.5	Figure 8.4 (bar chart)
Largest single fine	₹100.00	Figure 8.2
Total Total Copies card	475.00	Figure 8.4
Students by year (First, Second, Third, Fourth)	0, 1, 2, 2	Figure 8.4
E-mail subject received	New Book Transaction Alert	Figure 8.3

8.4 Limitations of the Low-Code Approach

The following constraints and issues were met while building this application.

- **Workflow debugging.** Several workflows named Check_Book_Availability existed while the no-copies rule was configured, and the errors could only be traced by opening each workflow and checking its actions. The problematic duplicates were disabled (BUG-01).
- **Validation limits.** A returned book is added to Available Copies without comparing it with Total Copies, so Logik der Forschung shows 12 available of

10 in Figure 8.1. A validation is proposed (BUG-05).

- **Fine reset.** The scheduled script sets the fine only when the overdue days are greater than zero and does not reset it when a transaction is no longer overdue (BUG-04).
- **Dashboard labels.** The dashboard components were kept as built, so the card in Figure 8.4 still reads “Total Total Copies” (BUG-06).
- **Lookup display.** The Book and Student lookups in Figure 8.2 show codes such as BC-001 and CODE1001 rather than titles and names. A formula or concatenated display field would be needed for a friendlier label.
- **Editions and trial limits.** The screenshots show a trial that expires in 3 days with an Upgrade option, so some governance features could not be assumed.
- **External service.** ISBN autofill depends on the Open Library service and fills only the book name, publisher and publication year. The author is entered manually.
- **Platform dependency.** The application exists only inside Zoho Creator and its logic is written in Deluge, which is specific to Zoho (vendor lock-in).
- **Scripting scope.** Deluge covered every need of this project, but its statement and API-call limits were not tested because the scripts are small.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application is **no-code configuration**: the four forms with their lookups, the Additional Authors subform, the reports, the dashboard page, the Admin, Librarian and Student roles with their permission sets, and the Book Issue Process Blueprint. **Low-code** was needed for the Deluge scripts: the ISBN autofill through invokeurl, the stock deduction, the return restoration with the Return Processed guard, the Calculate_Fine function, the scheduled Bulk_Overdue_Fine_Update, the transaction e-mail and the daily overdue reminder. **Pro-code** was not needed, because no Catalyst function, custom widget or Creator REST API client was built.

A move towards pro-code would be justified if the library wanted barcode or RFID scanning at the counter, an online payment gateway for fines, a dedicated mobile application for students, or integration with an institutional system, because these go beyond what forms, reports and short scripts can do comfortably.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Smart Library Management System shows how scattered library records can be replaced by a small low-code application. Four linked forms store books, students, book transactions and fine payments, and each transaction is linked to one or more books and to a student. Deluge scripts fill the book details from the Open Library service through invokeurl [5], [6], reduce Available Copies when a book is issued, restore them once when it is returned, and apply a fine of ₹50 for each overdue day. A scheduled batch update [8], [9], a transaction e-mail, a daily overdue reminder and the Book Issue Process Blueprint [7] carry the process forward, and the Admin, Librarian and Student roles control access. The reports and the dashboard give an immediate picture of the transactions, fines and copies.

The work also has limits that are recorded openly. Eighteen of the nineteen test cases passed and the zero-stock attempt (TC-07) was not recorded. Two defects remain open (BUG-04, BUG-05) and one dashboard label was accepted as built (BUG-06). Feedback from other users has not been recorded, no timing measurements were made, and the scripts were not tested against the documented Deluge limits [10]. Studies of low-code practice report that developers meet challenges with such platforms [1], [2], and this project met some of them on a small scale, such as the duplicate workflows that produced errors (BUG-01). These items are the first to address before the application is used for the records of a real library.

9.2 Future Enhancements

- Reset Fine Amount to zero in the scheduled update when a transaction is no longer overdue (BUG-04).
- Add a validation, in the form and in the return script, so that Available Copies cannot be larger than Total Copies (BUG-05).
- Record the result of a zero-stock issue attempt as evidence for the Block Issuance If No Copies workflow (TC-07).
- Collect feedback from librarians and students and record it in the user acceptance testing table.

- Rename the “Total Total Copies” card (BUG-06) and show book titles and student names instead of codes in the lookups.
- Use barcode or RFID scanning at the counter to select the book and the student.
- Add an online payment gateway so that a Fine Payment record is created when a fine is paid.
- Add a student portal or mobile access for borrowing history and due dates, which were not configured in this application.
- Extend the notifications, for example a reminder before the due date, and add more analytics such as the most borrowed books and department-wise borrowing.
- Add book recommendations for students based on the categories they borrow.

9.3 Learning Outcomes and Career Relevance

Building this application gave practice in the main parts of Zoho Creator [3]: data modelling with forms, lookups, a multi-select lookup and a subform; mandatory-field validation; reports and a dashboard; Deluge scripting [4] including an external API call, loops over selected records and a calculation function; a scheduled batch workflow; e-mail and reminder automation; a Blueprint; and roles with permission settings. It also showed how testing and debugging work in a low-code platform, including cleaning up duplicate workflows and adding a Return Processed guard to stop stock from being restored twice.

These skills relate to the roles of citizen developer, low-code developer, Zoho Creator and Deluge developer, solutions consultant and automation engineer. They also provide a basis for moving to pro-code extensions, such as barcode scanning, a payment gateway or a mobile application, when the requirements outgrow forms, reports and short scripts.

The project also showed that automation has to be checked against real records. Several problems, such as the repeated stock restoration (BUG-02), appeared only when the same record was saved again, so each rule needed a test with real data. Keeping a test case table and a defect log made these problems easy to trace, and the same habit will be useful in any future low-code or pro-code project.

REFERENCES

[1] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, “Characteristics and Challenges of Low-Code Development: The Practitioners’ Perspective,” in *Proc. 15th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2021. [Online]. Available: https://arxiv.org/abs/2107.07482 (Accessed: 1 Nov. 2026).

[2] M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, “An Empirical Study of Developer Discussions on Low-Code Software Development Challenges,” arXiv:2103.11429, 2021. [Online]. Available: https://arxiv.org/abs/2103.11429 (Accessed: 1 Nov. 2026).

[3] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 1 Nov. 2026).

[4] Zoho Corporation, “Deluge Script Reference.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 1 Nov. 2026).

[5] Zoho Corporation, “invokeUrl task for API calls,” Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/deluge-invoke-url.html (Accessed: 1 Nov. 2026).

[6] Open Library, “Developer Center: APIs.” [Online]. Available: https://openlibrary.org/developers/api (Accessed: 1 Nov. 2026).

[7] Zoho Corporation, “Understand stages and transitions,” Zoho Creator Help (Blueprint). [Online]. Available: https://www.zoho.com/creator/help/new-workflows/understand-stages-transitions.html (Accessed: 1 Nov. 2026).

[8] Zoho Corporation, “Understanding schedules,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/schedule-workflows/articles/understand-schedules (Accessed: 1 Nov. 2026).

[9] Zoho Corporation, “Create and manage batch workflows,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-batch-workflows/articles/create-and-manage-batch-workflows (Accessed: 1 Nov. 2026).

[10] Zoho Corporation, “Deluge limitations,” Zoho Creator Help. [Online]. Available: https://zoho.com/creator/help/script/usage-limits.html (Accessed: 1 Nov. 2026).

Note: only sources actually cited in the text are listed. No other statistics, quotations or literature are used in this report.